

Centrally Guided Formation Driving

David Follette '04 and Michael Yang '04

Submitted to the
**Department of Mechanical and Aerospace
Engineering
Princeton University**
in partial fulfillment of the requirements of Senior
Thesis.
May 3, 2004

Advisor: Professor Michael Littman
Reader: Professor Naomi Leonard
MAE 442 – Senior Thesis
106 pages

Acknowledgements

Professor Michael Littman

Our tireless and helpful advisor.

Professor Naomi Leonard

Our willing and helpful reader.

David Radcliff

For his assistance setting up and maintaining our lab station, and his tireless dedication to LabView and its unpredictable nature.

Ben Essenberg '05 (MAE)

For his work in the summer of 2003 that helped give us a starting point with the vision software for LabView 7.0.

Claude Samson

For his many papers that established the groundwork for our control algorithm, as well as his time and patience in explaining them to us.

Table of Contents

Acknowledgements	2
List of Figures	5
List of Equations.....	7
List of Tables.....	8
Abstract	9
Abstract	9
Introduction	10
Chapter 1: Project Goals & Overview.....	12
Chapter 2: Physical Implementation	13
Hardware	13
Software	16
Chapter 3: Vision: Locating and Tracking Multiple Vehicles	20
Locating the Vehicles.....	20
Tracking the Vehicles – Essenberg’s Algorithm	20
Locating Multiple Vehicles	21
Ordering Multiple Vehicles.....	22
Choice of Grayscale over Color.....	22
Chapter 4: Path Planning	25
Cubic Path Planning	25
Chapter 5: Analyzing Real Systems	34
A Brief Note about Terminology	34
An Intuitive Explanation for the Model.....	35
The System Model.....	35
Defining the Objective.....	38
Modeling the Physical System	39
Velocity	40
Steering.....	43

Building the Controller	45
First Approach: Approximate Linearization in Chained Form	47
Kinematic Equations	47
Chained Form	48
Linearization Control	49
Second Approach: Approximately Linearized System in Original Coordinates ..	55
Model Description	55
Application of the Algorithm	63
Finding the Gain Values k.....	64
Third Approach: Full State Linear Feedback	68
Conclusion	78
Chapter 6: Formation Driving	79
Traversing a Path	79
Following the Leader	82
Path Update	82
Path Jump.....	83
Following Error.....	84
Driving in Formation.....	86
Caravan	86
Geese Formation	89
Conclusions	92
Future Work.....	93
Tips, Tricks, & Recommendations	94
LabView.....	94
Appendix A: Cubic Path Planning Algorithm as Instantiated into LabView	95
Appendix B: Linear Control Algorithm as Instantiated into LabView	97
Appendix C: A Graphical Explanation of the LabView VI.....	100
Front Panel.....	100
LabView Image Description	102
Outside the Loop.....	102
Vision System	102
Car Controllers.....	102
Miscellaneous	103
Reference List	104

List of Figures

Figure 1: Installation of the Overhead Camera	13
Figure 2: Comparison of images with and without a lens	14
Figure 3: Geared motor Replacement	15
Figure 4: Geared motors with sleeves placed in the vehicles	15
Figure 5: R/C Controllers wired to the DAQ card	16
Figure 6: The White Circle/Black Circle Tracking Method	17
Figure 7: Raw video image with cross hair overlay	18
Figure 8: White highlighted image from which the computer finds the car and spot, overlaid with cross hairs	18
Figure 9: Camera recorded path of a remote controlled car	19
Figure 10: Finding Multiple Cars using "Black Out"	22
Figure 11: Four R/C Cars tagged in black, blue, green, and red.....	23
Figure 12: Four R/C Cars viewed in grayscale from darkest to lightest (L-R).....	23
Figure 13 : Cubic Path for (x) and (y) individually	27
Figure 14: Cubic Paths synchronized for Cartesian coordinates	28
Figure 15: Cubic Paths for varying t_f	29
Figure 16: t_f calculation comparison, start and end at 10 degrees	31
Figure 17: t_f calculation comparison, start and end at 80 degrees	31
Figure 18: t_f calculation comparison, long distance, start at 10 and end at -50 degrees...	32
Figure 19: DeLuca, Oriolo, and Samson's diagram of the state space variables.....	35
Figure 20: Simulink model of the plant (R/C car)	36
Figure 21: Simulation of speed and steering step response	37
Figure 22: Diagram of a one error variable system	38
Figure 23: Inability to superimpose step responses with only distance error as the error variable.....	39
Figure 24: General control loop for the system	39
Figure 25: Orthographic view of trajectory	40
Figure 26: V1 vs. Time	41
Figure 27: V2 vs. Time	41

Figure 28: Speed vs. Throttle Voltage	42
Figure 29: Forward speed vs. Throttle Voltage	42
Figure 30: Backward Speed vs. Throttle Voltage	43
Figure 31: Turning Radius as a function of steering voltage.....	44
Figure 32: Turning Diameter as a function of v_2 (car model).....	44
Figure 33: Control Inputs to Steering Voltage.....	45
Figure 34: Rear Wheel Drive Kinematic Model Illustration	46
Figure 35: Car Model.....	48
Figure 36: Simulink realization of Linearized Chained-form Controller	53
Figure 37: Simulink output for Chained Form Controller	54
Figure 38: Errors vs. Time for Chained-Form Linearized Controller	55
Figure 39: Rotated first turn.....	65
Figure 40: Orthographic view of step response	66
Figure 41: Step response as a function of time	66
Figure 42: Modeled and Actual Step Responses (blue = modeled, red = actual).....	67
Figure 43: Simulink implementation of full state exact linear dynamic feedback controller	68
Figure 44: Target Trajectory	71
Figure 45: Target Trajectory with Realization superimposed	72
Figure 46: Errors vs. Root Location, k	72
Figure 47: Non-well behaved v_2 input	74
Figure 48: Well behaved v_2 input.....	74
Figure 49: Near optimal v_2 vs. time input	75
Figure 50: Trajectories for $[-3 \ 30 \ -3 \ 100 \ 3]$	75
Figure 51: Path trajectories with optimal parameters	77
Figure 52: Off-trajectory simulation.....	78
Figure 53: Traversing a Path Screenshot	80
Figure 54: X and Y Errors for path following at low speed	80
Figure 55: Theta error for path following at low speed	81
Figure 56: X and Y Errors for path following at high speed	81
Figure 57: Theta error for path following at high speed	82

Figure 58: Dynamic path generation from one vehicle to another	83
Figure 59: Convergence test for caravan formation - start	88
Figure 60: Convergence test for caravan formation - end	88
Figure 61: Distance error as a function of time for caravan formation	89
Figure 62: Orientation error as a function of time for caravan formation	89
Figure 63: Geese Formation approaching steady-state	90
Figure 64: Geese formation with large initial error.	90
Figure 65: LabView VI Image	101

List of Equations

Equation 1: Conversion of RGB to Y (of YUV)	24
Equation 2: Basic form of a cubic function	26
Equation 3: Determination of Constants for Cubic Path	26
Equation 4: Second derivative of the generic cubic function	30
Equation 5: Second derivative of the path, differentiated by t_f and set to zero.....	30
Equation 6: State space for plant (R/C car)	35
Equation 7: State Space for Plant (R/C Car).....	48
Equation 8: Chained Form Relationships	49
Equation 9: Change of coordinates for controller.....	49
Equation 10: Calculating phantom input from time-invariant path coordinates.....	50
Equation 11: Linearizing form.....	50
Equation 12: Linearizing form without disturbance	50
Equation 13: Linearizing form for desired state	50
Equation 14: Error Equation	51
Equation 15: Computing matrices $A(t)$ and $B(t)$	51
Equation 16: Linear time-varying error function for controller.....	52
Equation 17: Design of Control Variables.....	52
Equation 18: Closed Loop System Matrix.....	52
Equation 19: Hurwitz Stability Polynomial.....	52
Equation 20: Kinematic Model for Control and Reference Vehicles	56
Equation 21: Settings for w_1 , w_2	58

Equation 22: Error Equations.....	58
Equation 23: w_1 and w_2 optimal settings for Lyapunov function.....	60
Equation 24: Settings for v_1, v_2	60
Equation 25: Settings for v_1, v_2 to guarantee convergence.....	61
Equation 26: Fully Linearized, time-invariant time-scaled error system	63
Equation 27: Gains as determined by time-scaled system.....	63
Equation 28: Reduced Complexity settings for v_1, v_2	64
Equation 29: Full State Linearization Equations for Car in Chained Form.....	69
Equation 30: Full State Linearization for Car in non-chained (regular) form	69

List of Tables

Table 1: Advantages and disadvantages for various control scenarios.....	47
Table 2: Optimal parameter values to optimize each state variable's error	76

Abstract

Controlling vehicles in a fleet via central command has many advantages over peer-to-peer networks, including improved route planning and a cheaper implementation, since the individual cars can remain unintelligent. As cars become more and more intelligent, there will be a greater need for control, tracking, path planning, and coordination among vehicles, which is the focus of our project.

Our goal for this project was to create a working system that integrated vision tracking, path planning, and modern control for rear-wheel drive vehicles. Our system effectively tracks four separate vehicles and drives them in various formations behind a leader car controlled by a human being. To drive in formation, each of the three following vehicles is given a reference path to its position in the formation. Updated every half of a second, the reference path guides the cars to their formation and then maintains the formation. The paths are calculated as a cubic polynomial of two variables, parametrized by time. The control algorithm is a linearized controller with four state variables – x and y position, heading angle and steering angle, and two output variables – steering and throttle. The system is implemented in hardware with a firewire camera and four remote controlled vehicles connected to a PC.

Finally, the LabView implementation has been written in modular method that lends itself to updates and can easily become a test bed for algorithmic improvements in the future.

Introduction

Human beings are not traditionally mobile animals, since we neither run fast, swim fast, nor fly at all. However, through the use of technology, we have increased our mobility, and we are facing many of the same transportation problems that have challenged animals for millions of years – how to move in groups quickly and efficiently. When birds and fish travel in groups, they travel in a formation, in order to reduce the effects of aerodynamic drag, making the group more efficient. In similar ways, we would like our networks of cars, boats, and airplanes to benefit from these formations.

Although formations offer great benefits in efficiency, it is very costly to mimic animals and put an animal's machine equivalent of the brain and sensory abilities into mechanical transportation devices. It would be more efficient to concentrate the intelligence of the group into one centralized location, and have this system command each of the individual members. Then, each would not need to be particularly intelligent, or in the engineering sense, not carry as expensive of a computer on board. In addition, the central system could have a global knowledge of the complete route from start to finish and effectively devise the most efficient path while accounting for obstacles along the way, since it has more information than any of the individuals alone.

A number of other “intelligent” or “platoon” driving and flying systems incorporate significant numbers of sensors, cameras, and computers into each of the individual vehicles. Many of these systems incorporate intelligence into each vehicle since they would like them to be able to function independently of their platoon. Some other systems also use GPS technology to get a global position for vehicles and help them plan their routes, but few use central command for multiple vehicles.¹ Compared with a centrally commanded system, the vehicles in these systems must rely on each other for accurate information, they do not receive the benefits of a holistic viewpoint, and they are typically expensive.

For our senior thesis project, we designed and implemented a centrally guided visual system that controls remote controlled cars in formation driving. The central computer system collects information from all of the cars via a camera and outputs commands to the unmodified remote controlled cars only in terms of steering and speed

control. We have successfully built a central system that autonomously tracked and controlled up to four remote controlled cars in a variety of different formations.

In the future, we see the applications for this project in areas other than passenger vehicles. With passenger vehicles, on-board vision and sensing remains important for immediate concerns, such as collision avoidance and unexpected problems. In addition, with a huge network of people and cars, a central computer faces significant challenges with susceptibility to malicious software, and the great responsibility with controlling vehicles with people inside. However, a centrally controlled system can be very effective in small, closed systems, where all the vehicles are unmanned, and all owned and maintained by the same entity, such as in a factory, or in space exploration. In the future, the optimal solution also most likely involves a hybrid of central control and on-board systems², where a global, central system gives an efficient, holistic view of the route and formation, while the local systems work as a safety backup, able to protect vehicles should the central system malfunction. We see our project as an integral part of that development, strengthening the understanding of global route planning, and developing the central navigation system.

Chapter 1: Project Goals & Overview

Formation driving requires a number of different systems to work together simultaneously to be successful. The first segment of the process is gathering the information on the cars from the camera. Next, the computer processes the data, calculates the necessary changes that need to be made to the vehicles' controls, and outputs the commands via radio control. Each one of these processes follows a number of different rules and algorithms to allow the cars to move smoothly and efficiently.

Although our final goal was formation driving, we set a number of intermediate goals along the way. Our first challenge was to design a control algorithm and setup a computer simulation of a single vehicle. Next, we began to build the hardware and set up the lab with a camera, a computer, and four remote controlled vehicles. In addition to putting the hardware together, we interfaced the computer with the hardware so that they could all communicate reliably. Next, we worked on path planning that would give us a smooth path between arbitrary points on the screen, and devised methods to keep the cars in formation. Then, we implemented our simulations in real life and had one car move from its current position to an arbitrary point on the screen. Finally, we expanded our program to multiple vehicles and gave them formations to follow.

Ultimately, we built a system that can track up to four vehicles, and control three of them to follow a leader vehicle in formation. Our goal in this project was not to build a system with the absolute best control algorithm, path planning algorithm, or vision system, since each of these could be a separate Senior Thesis in itself. Instead, we wanted to learn about the necessary features of a full feedback vision system, and implement algorithms that work effectively. In addition, we wanted to build a system that was modular and easy to update, such that students in future years could work to optimize each individual area, and have a robust test bed to test their programs. We feel that coordinated systems of vehicles is on the forefront of mechanical engineering research, and that our project will lay the groundwork for many students in the future.

The format of this paper will follow the path of the information in the control system, from the point of collection via the overhead camera, to the final radio controls sent to the individual vehicles.

Chapter 2: Physical Implementation

One of our early tasks was to implement all of the hardware that we would need for formation driving. In addition to ensuring that the hardware worked, we needed to determine the constraints of the hardware, especially the vehicles, the camera, and the computer system.

Hardware

The setup for our lab station includes an “iBrain Fire-i” FireWire camera attached to a 15 foot ceiling, giving us a visible area of 10 feet by 6 feet on the ground. (See Figure 1) The camera is connected to a 3.0 GHz Pentium computer, running LabView, which processes the images and controls the vehicles.

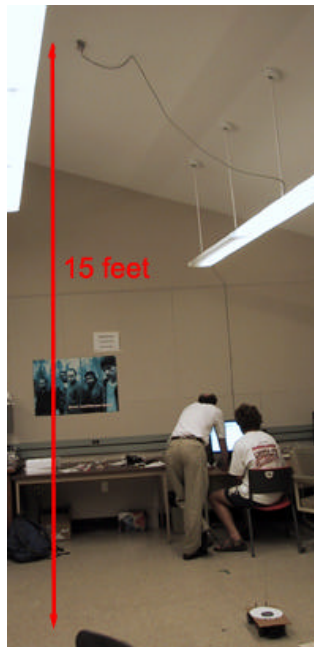


Figure 1: Installation of the Overhead Camera

After some testing, we found that the cars (approx. 6” by 12”) cover a fairly large area, and have a minimum turning radius of about two feet. Therefore, we decided to add a lens to the camera to increase its visible area. With the addition of a negative lens, we increased the visible area from 10 feet by 6 feet to 13.5 feet by 10 feet, or an increase in area of 125 percent. (See Figure 2) We used a relatively large lens, such that the

distortion is not noticeable to the human eye, and will not cause a significant error to the control system.

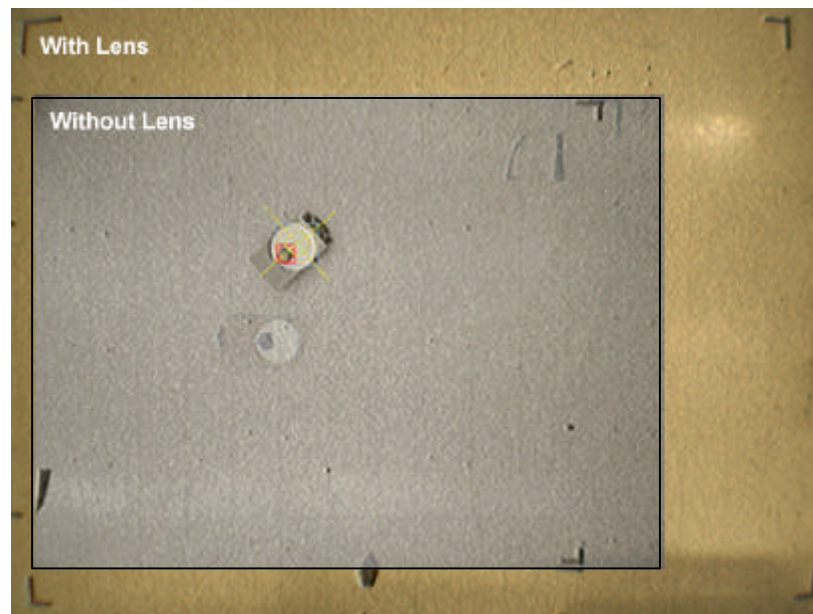


Figure 2: Comparison of images with and without a lens

The vehicles are Tamiya 1/10 scale models of Porsche Boxsters. We chose model cars so that we could be familiar with all of their working parts, and so that we would have the flexibility to change out the parts or modify them to meet our needs. To maintain the no-slip condition, to ensure that 30 frames per sec would be sufficient to control the vehicles, and to stay within the area of a classroom floor, we needed the vehicles to move at a very slow, constant speed.

After some testing, we found that the stock motors had difficulty maintaining a constant low speed, especially when turning. The Tamiya models are shipped with large, high-speed motors that are geared with a ratio of almost 1:1. When the front wheels are turned, the motor must work slightly harder to turn the vehicles, and the large motors do not have the necessary torque at very low speed. Therefore, we modified the cars to take a much smaller geared motor that output a higher torque at a lower speed. We purchased miniature gear motors from MFA Como Drills, located in the UK, but they were much smaller than the stock motors for the Tamiya model cars. (See Figure 3) So, we machined sleeves for the motors that allowed them to fit into the cars, and the vehicles

now operate with a much smaller motor with a 30:1 gear reduction. (See Figure 4) Since the motor outputs much greater torque than before, the open-loop speed control is much more predictable, and we will not need a controller to maintain a constant speed.



Figure 3: Geared motor Replacement

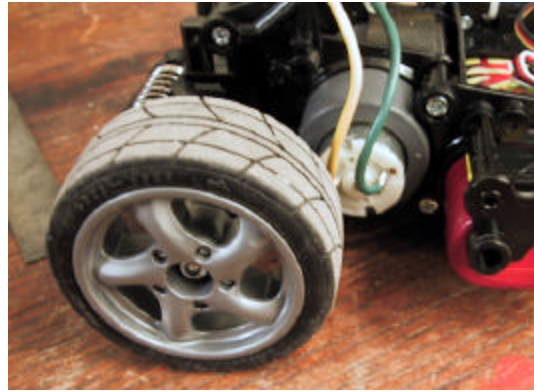


Figure 4: Geared motors with sleeves placed in the vehicles

To transmit commands to the cars, we had to interface the computer with the remote controllers. Instead of building our own transmitters and receivers from scratch, we simply modified the transmitters that came with the vehicles. The controls for steering and velocity were originally just potentiometers, so we removed them and replaced them with a wired connection directly to the “Data Acquisition Card” (analog output), connected to the computer. This interface allows LabView to output voltages to the transmitters, which will output signals to the remote controlled cars. (See Figure 5) The first successful test was to drive the cars via the computer, which we completed in early October.

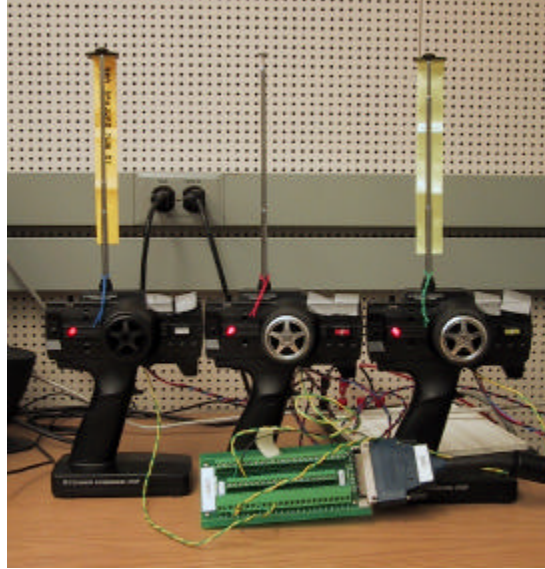


Figure 5: R/C Controllers wired to the DAQ card

Software

We used LabView to implement the vision and the controller because it can accommodate MATLAB scripts, C code, and because it contains powerful image processing functions. All of the inputs to the computer (and controller) are via the iBrain FireWire Camera, capturing 640x480 pixel images at 30 frames per second. Our vision program is based on a vision tracking implementation written by Ben Essenberg '05 (MAE) over the summer of 2003. To track his vehicle via camera, he tagged it with a large white circle with a smaller non-concentric black circle inside, which would define the orientation of the vehicle. Figure 6 below shows our own white and black circles mounted on one of our R/C cars. The positioning of the spot was determined by the needs of the control algorithm, which uses the location of the center of the rear axle as its reference point. Therefore, the middle of the white circle is placed directly over the center of the rear axle, and the dark spot is oriented straight ahead.

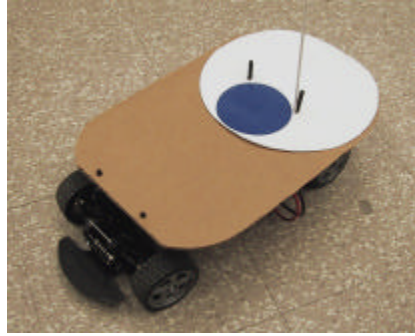


Figure 6: The White Circle/Black Circle Tracking Method

Essenberg's LabView program is able to locate this white circle within the visible area of the camera, and then locate the dark circle within the white circle and determine the orientation of the vehicle. It also tracks the motion of the vehicle by calculating its velocity, predicting its future location, then finding the vehicle in the next frame. We modified the code slightly to remove some extraneous features and improve the response time of the search. We also expanded the capabilities of the program to identify different colored circles, and therefore, track multiple cars. As is, Essenberg's program takes the image from the camera and outputs the (x, y) coordinates of the center of the circle, the heading angle of the vehicle, and its velocity. Figure 7 is the raw camera image from above, with the computer generated cross hairs tracking the vehicle. Figure 8 is the processed image that highlights white areas and locates the black circle within the white area to determine the orientation of the vehicle. This image also has the tracking cross hairs overlaid.

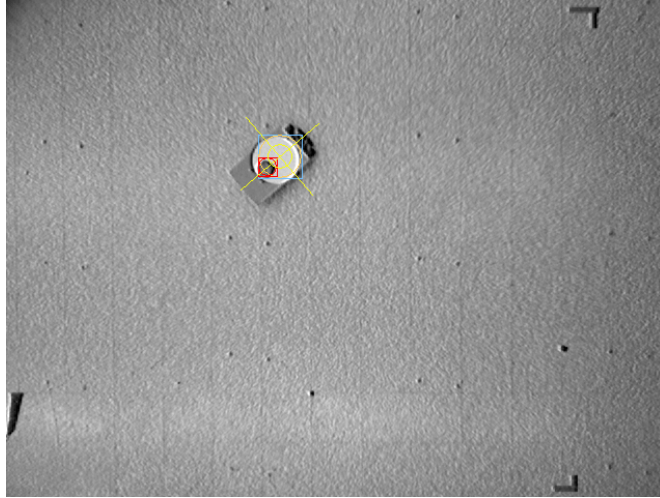


Figure 7: Raw video image with cross hair overlay

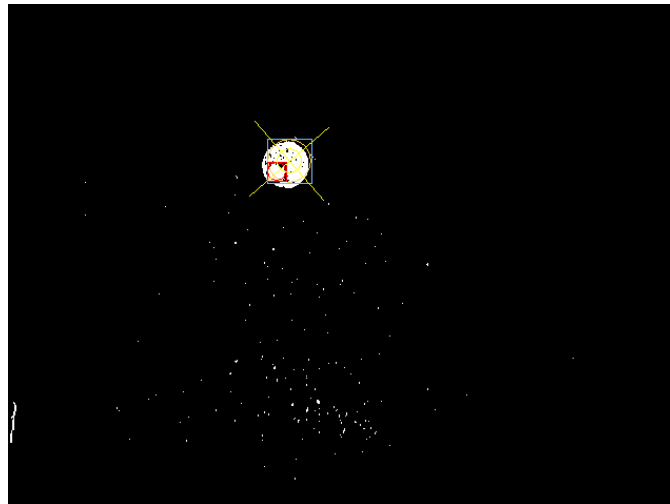


Figure 8: White highlighted image from which the computer finds the car and spot, overlaid with cross hairs

One of the first uses of the overhead camera and LabView was to record data on the vehicles. Figure 9 is a plot of the car's position (parameterized by time) as we drove it in circles on the floor. It is interesting to note that although the steering angle was kept constant, the circles are not concentric, showing some slippage of the wheels during turning. It does show, however, that the refresh rate for the camera is high enough for us to get smooth data on the position of the vehicle.

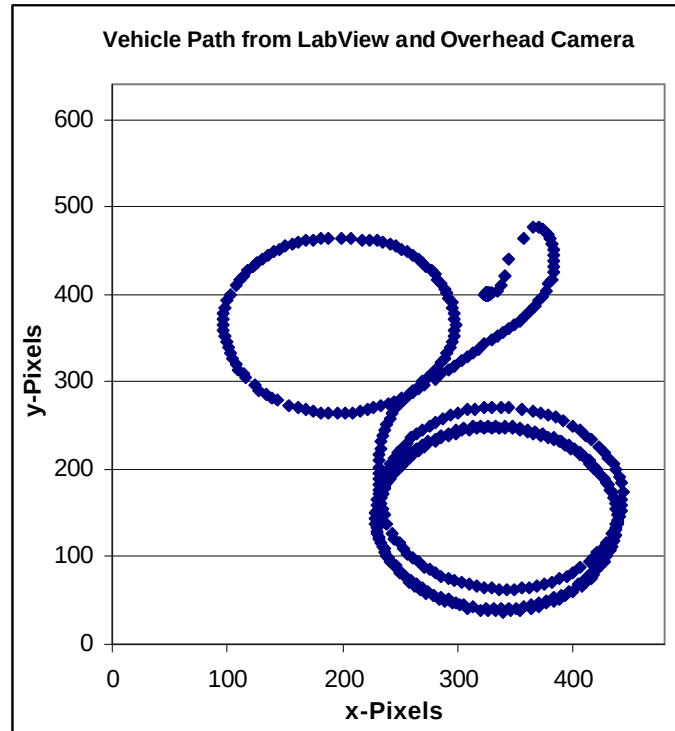


Figure 9: Camera recorded path of a remote controlled car

Chapter 3: Vision: Locating and Tracking Multiple Vehicles

Given a camera that outputs 30 grayscale frames per second, cars that have white circles on them with non-concentric colored circles, and a tracking program, we implemented a method to locate, track, and identify four independent vehicles simultaneously.

Locating the Vehicles

The first step to locating the vehicles within the huge 640 by 480 grid is to take advantage of their large, white, identifying circles. (See Figure 6 and Figure 7 for the target design and for the image that the camera sees of a single vehicle on the floor)

Taking the image from the camera, the first step is to threshold the image so that areas of the image that are brighter than a certain threshold are made completely white, and anything darker than a certain threshold is made completely black. (See Figure 8) With the white areas explicitly revealed, it is easy to identify the white spot on the vehicle within the whole screen. At this point, the algorithm uses the function, “Complex Points” to find the largest white area in the screen, and create a “Region of Interest” about the area. We named this function “Find Car”. “Region of Interest” (ROI) is a LabView variable type that allows the user to concisely describe a section of the image, which we use to remember the location of the white circle, and track it in space. The location of the large white area is sufficient information to pass on to Essenberg’s algorithm to track the vehicle as it moves around.

Tracking the Vehicles – Essenberg’s Algorithm

Essenberg’s algorithm takes the ROI of the large white circle and isolates that region of interest within the large image. The algorithm takes this area and inverts the image, making the large outer circle black, and the dark inner spot, white. Then, it repeats the process of finding the location of smaller white circle within a large black area. This program outputs the locations of the center of the larger white circle, the center of the smaller dark circle, and the global theta angle (orientation) of the vehicle, based on the relationship of the light and dark centers. To follow the vehicles in time, Essenberg’s algorithm first compares the current position of the center of the vehicle with its position

at the last iteration and calculates a velocity. Using this velocity, it predicts where the vehicle will be in the next iteration and perturbs the regions of interest accordingly to continue tracking the vehicle. These regions of interest are a system of masks that greatly reduce the amount of information that must be retained iteration to iteration. However, since the algorithm is only able to update every 30th of a second, the algorithm can only track vehicles of a limited speed. The algorithm had some difficulty tracking the cars before we geared down our motors, but the slower cars do not exceed the capabilities of the tracking algorithm, and we have had no problems with it.

Finally, the algorithm overlays a cross-hairs over the image at the position of the car to give a visual indication of the vehicle tracking.

Locating Multiple Vehicles

Locating a single vehicle on the floor is simple, since it is the largest white object on the whole screen. However, with multiple vehicles there are four separate white circles to find and track.

Before we began, we were aware that the tracking algorithm was very robust in the sense that it is difficult to disrupt once it finds a target to track. The presence of other targets on the screen, or other random objects did not distract it from tracking its original target, since it follows the object it is tracking, rather than continuously searching the screen for a large white spot. Therefore, the challenge was to find a way to initially locate four cars simultaneously, without tracking any one of them twice.

To find the first car, we used the original “Find Car” function, which captures the car with the largest white area on the screen and passes on its region of interest. (Since each vehicle has the same size white circle, the one it happens to choose is arbitrary) At this point, if we run “Find Car” again with the same image, it might find a different car, or more likely, it will end up capturing the same car again. Instead of modifying the “Find Car” function, we wrote another function called “Black Out” that takes the region of interest from the first “Find Car” function (the location of the car), and paints a black square of the size of the car, in that region. This blacked out image is then passed on to the next “Find Car” function call so that the first car will be invisible to the second “Find Car” function. We repeat this process two more times to find the third and fourth cars in

the plane. See Figure 10 for a series of the thresholded images with one car blacked out at a time.

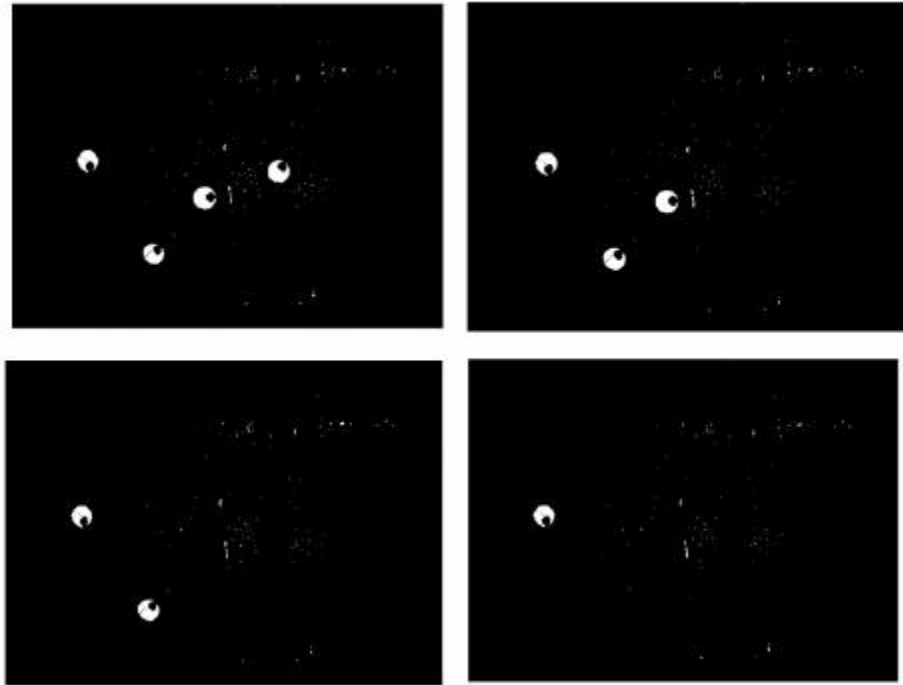


Figure 10: Finding Multiple Cars using "Black Out"

Ordering Multiple Vehicles

With the process of locating and tracking each of the vehicles complete, we needed to determine which was which, so we could control them appropriately. For the four vehicle case, each of the vehicles was outfitted with a white circle and a different colored dark circle within the white circle.

Choice of Grayscale over Color

To keep up with the 30 frames per second refresh rate on the camera, it was necessary to keep the image processing as short and simple as possible. One way to track the individual cars would be to give them each a colored spot, and capture a color image to separate out the spot. Most color images are stored in RGB (red, green, blue), so it would be relatively simple to pick out the solid red, green, blue, or black, inner circle. However, processing a color image increases the amount of image information by a factor of 3

(without compression), and it slowed our LabView program to 15 frames per second, which was unacceptable. A better solution was to use the grayscale image from the camera and to differentiate the inner circles by different shades of gray. The grays are represented in 8-bit depth, so with a range of 256 gray values, it is feasible to identify four unique shades. We decided to continue to use the colored inner circles for our own sake, since it is much easier for us to identify separate colors, rather than shades of gray, and the colors already show up as different shades of gray on the grayscale image. Therefore, we printed white circles with colored inner dots for each of the cars in red, green, blue, and black, and processed a grayscale image. See Figure 11 for a picture of the four cars in color, and Figure 12 shows the images in grayscale, as they would be viewed by the camera. Notice that the colors are differentiable by shades of gray.

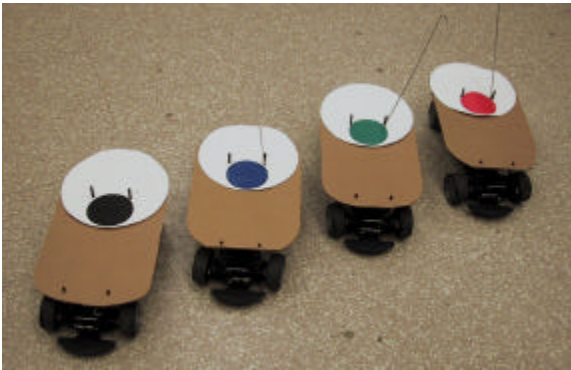


Figure 11: Four R/C Cars tagged in black, blue, green, and red.



Figure 12: Four R/C Cars viewed in grayscale from darkest to lightest (L-R)

To order the four cars, we take the outputs from the find and track cars algorithms and determine the pixel location of the center of the darker spot on each car. Using the IMAQ GetPixelValue, we determine the grayscale value of each spot, and pair that with the data on its position and orientation. Then, we order the grayscale values and pair each position and orientation with the correct color car. Intuitively, we determined that in order from darkest to lightest is black, blue, green, and red. The standard conversion scheme used to convert an RGB image to a YUV image (where Y is a measure of luminance) follows Equation 1.

$$Y = (0.257 * R) + (0.504 * G) + (0.098 * B) + 16$$

Equation 1: Conversion of RGB to Y (of YUV)

This shows that the order of darkness should be black, blue, red, green, since green will provide the most luminance. However, our empirical tests with our targets show an order, from darkest to lightest, of black, blue, green, and red. This discrepancy is most likely due to differences in the color laser printer's toner colors. Nonetheless, the colors follow a consistent grayscale ordering, so we are able to effectively identify and track the four vehicles.

Chapter 4: Path Planning

Cubic Path Planning

The simplest method of traveling between two points is to create a line between them. However, a useful path also takes the vehicle from its current position and orientation and ends in a new position and orientation. A line may sometimes accomplish this task, but does not explicitly account for beginning and ending orientation, which are important factors. One of the best examples of a challenging path planning task is parallel parking. The description of the problem is simple: move the car 15 feet to the right. However, this requires a complex set of motions, since the car is underactuated, with nonholonomic constraints.

Path planning takes these factors into account and produces a path that the car can follow, given its nonholonomic limitations.

There have been a number of different methods researched of creating the “best” path between two points, while maintaining orientation at both ends. Some use cost functions to minimize the total distance, while others use complex splines to create smooth paths. Our goal was to begin with a very simple path planning algorithm that would create a smooth path between two points, preserving starting and ending orientation. The path should also be viable, given the car’s turning limitations, but this is not a critical aspect of the algorithm, as we will see in the usage of the planned path.

The simplest method is to use a cubic function, which allows up to two full turns between the initial starting point and the ending point, since a third degree function can have up to one change in sign of its second derivative over the desired range. With one convex and one concave turn, it is possible to go from any orientation at the beginning to any orientation at the end.

Our first thought was to simply derive a function based on two points and the desired first derivatives at these points. These four constraints would give rise to a cubic polynomial function. The basic form of a cubic polynomial for one variable (vs. time) is of the form in Equation 2.

$$q(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0$$

Equation 2: Basic form of a cubic function

Following the example in Craig's *Introduction to Robotics: Mechanics and Control*, the constants a_0 through a_3 can be determined to constrain the starting and ending positions, as well as the derivatives (with time) at the initial and final positions.³ The values are outlined in Equation 3.

$$\begin{aligned} a_0 &= q_0 \\ a_1 &= \dot{q}_0 \\ a_2 &= \frac{3}{t_f^2}(q_f - q_0) - \frac{2}{t_f}\dot{q}_0 - \frac{1}{t_f}\dot{q}_f \\ a_3 &= -\frac{2}{t_f^3}(q_f - q_0) + \frac{1}{t_f^2}(\dot{q}_0 + \dot{q}_f) \end{aligned}$$

Equation 3: Determination of Constants for Cubic Path

The constants are determined based on an initial and final position (θ_0 and θ_f), and initial and final derivative with time ($\dot{\theta}_0$ and $\dot{\theta}_f$). However, a simple cubic function that passes through two points with the desired orientation does not uniquely determine a path between them. Each of the possible paths between two points (preserving orientations) will have a different degree of curvature and a different overall length. Therefore, there is an additional parameter, t_f , which is a factor for the total time required to traverse the path. Any factor for t_f will be a valid solution through the two points, but the length of the path (and therefore, the curvature), will be different. To create a path for vehicles with a limited turning radius, it will be necessary to be able to limit the curvature of the path to something that does not exceed the limitations of the vehicles. This problem will be discussed in more detail later on.

For a vehicle to move around on a Cartesian plane, it must move in a smooth path of two variables, as a function of time. Since the vehicle will need to move in any direction, including horizontally, vertically, and in a circle, its path cannot be described by a simple function $y(x)$. Therefore, we created a cubic path for x and y separately as functions of time, then synchronized them in time to create the (x, y) path for the vehicle.

See Figure 13 for the (x) and (y) paths parametrized separately, then combined in Figure 14 for an example of the path recombined to create a path on the Cartesian plane. (The starting coordinates were (50, 10, -10) (x, y, theta), and the ending coordinates were (350, 300, -10).)

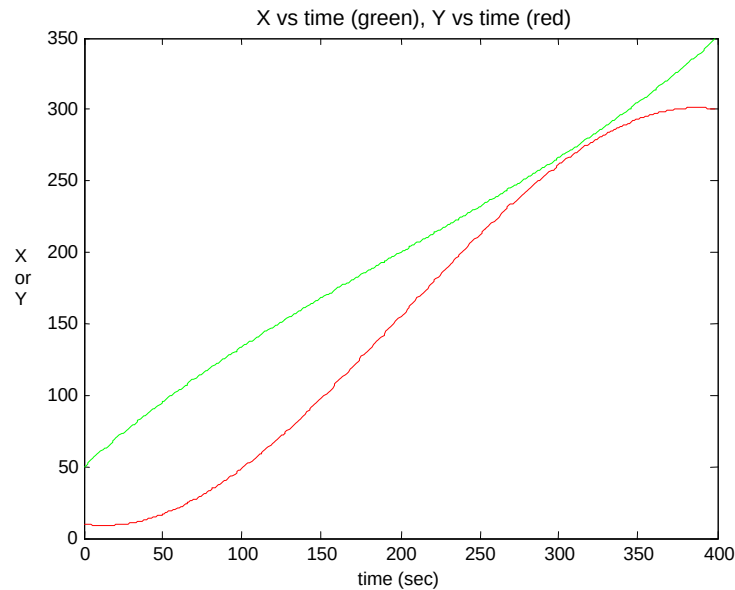


Figure 13 : Cubic Path for (x) and (y) individually

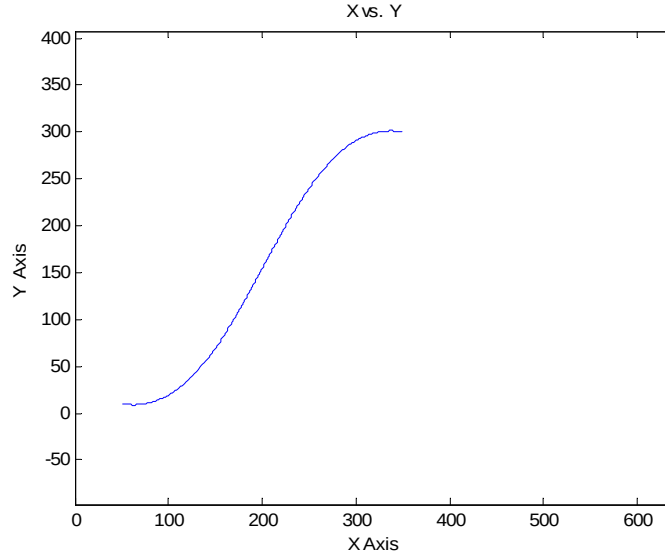


Figure 14: Cubic Paths synchronized for Cartesian coordinates

Looking at these results, it is easy to see potential situations where the path between the current position and the desired position could result in a path with very tight (unattainable) turns. Thus, we must find a way to limit the minimum turn diameter of the cubic path.

First, we returned to the problem of multiple cubic solutions to look for a solution. We experimented with simply making changes to the value of the time constant, and for some cases, the path will improve significantly, but for other cases, it can make the path significantly worse. The value of t_f generally affects the overall length of the path, which can increase the minimum turning diameter, but can also stretch the path out of control. Figure 15 uses the same start and end conditions as above, but for varying time constants. The blue path has a time constant of 100, the cyan path, 300, the black path, 800, the green path, 1600, and the red path 417, equal to the absolute distance between the starting and ending points.

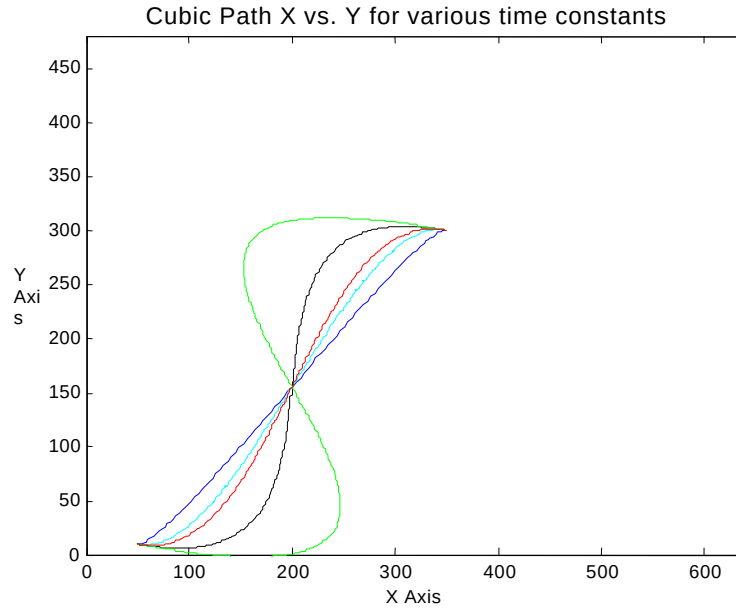


Figure 15: Cubic Paths for varying t_f

We initially conjectured that turning radius was equal to the curvature, or the second derivative of the path. For cubic functions, the second derivative is a linear function of time, and therefore, the local maxima will be at either time zero, time final, or both. A better example is to imagine a parabola. For a second-order function, the first derivative (slope) will be a line, and the second derivative (curvature), will be a constant. If a car were to drive along the path of a parabola, the amount that it turns the steering wheel is not constant over time. At the maximum (or minimum) point, it becomes necessary to complete a U-turn and turn the vehicle around, while far away from the maximum (or minimum), the car would be moving more or less straight ahead. Therefore, the second derivative is not quite the right measure of the turn radius of the function.

However, imagining a parabola of the function $y=x^2$ and $y=3x^2$, it is clear that the $y=3x^2$ case will come to a much narrower point at the focus. Thus, although the max or min of the second derivative is not relevant, limiting the value of the second derivative is important.

Differentiating our initial general cubic function twice gives rise to Equation 4.

$$\ddot{q} = a_3 q + a_2$$

Equation 4: Second derivative of the generic cubic function

Now that we have a function for the value of the second derivative, we want to minimize its value as a function of t_f . Theta in this case is a constant, (it will be something between 0 and 640, given our coordinate system) so only the sum of a_3 and a_2 must be minimized. Differentiating the second derivative as a function of t_f and setting the result equal to zero gives a solution for t_f , Equation 5.

$$t_f = 3 \left(\frac{\dot{q}_f - \dot{q}_0}{\dot{q}_f + \dot{q}_0} \right)$$

Equation 5: Second derivative of the path, differentiated by t_f and set to zero

It is immediately obvious that in some cases where theta-dot initial and final are zero (horizontal or vertical beginning and ending, for y and x, respectively), t_f will go to infinity. Ignoring that point for the moment, there is also one value for t_f that will be calculated for each of the x and y paths individually, before they are combined. To understand the performance of these two factors, we ran a number of Matlab simulations comparing the t_f values determined from the x path, the y path, and by the value determined by the absolute distance between the starting and ending points.

First, we tried starting and ending at 10 degrees while traversing diagonally along the plane. (See Figure 16) The x path minimized second derivative (Xmin) performed very well, since the beginning and ending slopes (in terms of x) were large for the x only trajectory. The opposite was true for the y case, where to minimize the second derivative of y, the path had to be much longer to minimize the second derivative.

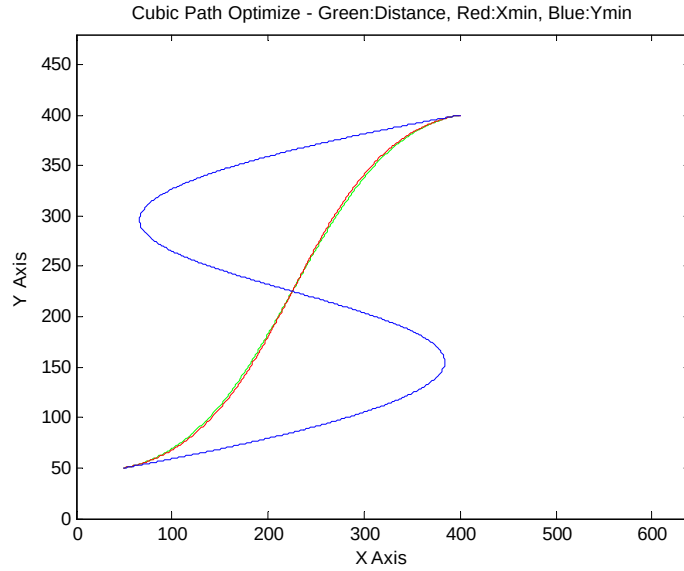


Figure 16: t_f calculation comparison, start and end at 10 degrees

Next, we tested the case where the start and end conditions were both at 80 degrees to see the effect reversed between x and y. (See Figure 17) Again, the time constant equal to absolute distance was just as good as the best minimized x or y alone.

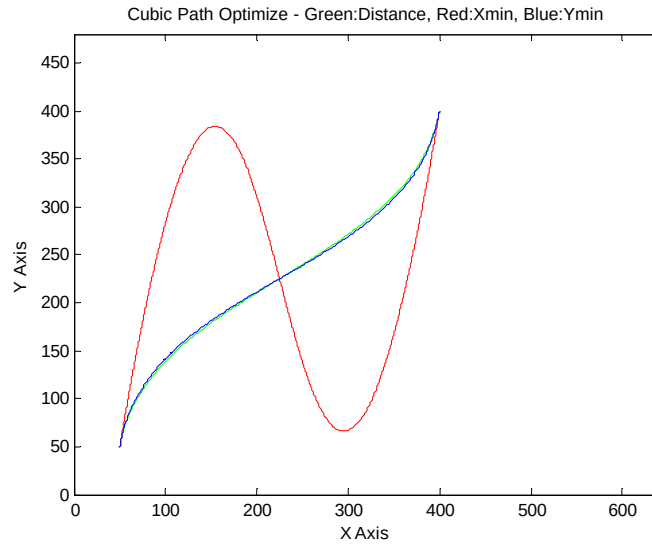


Figure 17: t_f calculation comparison, start and end at 80 degrees

With these results, it is clear that the absolute distance is at least as good as any minimization of the second derivative. The absolute distance also does not have the singularity where the starting and ending derivatives are both equal to zero, and it limits the total length of the path such that the path is very unlikely to leave the plane, assuming that starting and ending points are in the plane. The x and y minimization techniques are prone to creating very large time values that could leave the plane and cause the vision system to lose the vehicle. Figure X shows a long traversal across the plane with starting condition at 10 degrees and ending at -50 degrees.

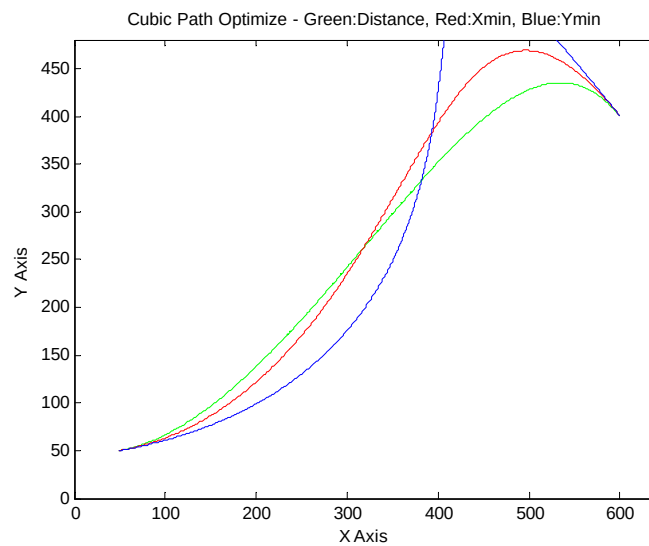


Figure 18: t_f calculation comparison, long distance, start at 10 and end at -50 degrees

In this instance, the XMin (red) appears to perform slightly better than the green, but the difference is not enough to warrant the concerns with singularities and the potential to have paths that leave the screen space.

Experimentally, we found that the best value for t_f is proportional to the actual distance between the two points, so our code calculates the absolute distance between the two points and uses the pixel distance value as t_f for the calculations.

Once the function for the path is determined, there is one additional step to creating the path that the car will actually follow. Since the vehicle controller tracks points (instead of velocity), it will need to be given a new point every cycle, since the car is moving forward. To maintain a constant speed, the points must be spaced not by a

constant “t” time step in the algorithm, but by determining the rate of change of the function, and stepping “t” appropriately. The function calculates the derivative of $x(t)$ and the derivative of $y(t)$ at each point, takes the magnitude of their sum, then scales the time step to maintain a constant velocity. The result is an array of (x, y) positions, with a time step of 0.033 seconds for 30 frames per second.

The function that we use to produce the path for the vehicles also takes a desired velocity as an argument, and places the points of the path the correct distance for a car, traveling at the desired speed, updating at a rate of 30 times per second. See Appendix A for the Matlab code for our path planning algorithm.

Chapter 5: Analyzing Real Systems

A Brief Note about Terminology

Real vehicle systems can be placed into two categories: fully-actuated systems and underactuated systems. A fully-actuated system is strictly defined as a system where the number of degrees of control is greater than or equal to the degrees of freedom, or degrees necessary to define their state. An underactuated system is where the degrees of control are fewer than the degrees of freedom. Satellites and spacecraft have 6 degrees of control and 5 degrees describing their state, making them fully-actuated. (i.e. – they can be controlled to move in any of the free directions, but to define their position only requires x , y , z , and two angular measures). However, cars have two degrees of control (steering and velocity), but three degrees are necessary to uniquely define their state (x , y , and heading angle), and hence, are underactuated systems.⁴

Real vehicle systems can also be categorized by the way in which they move in space. Vehicles with holonomic constraints can move in any direction on a plane, or in space, depending on whether or not they are land-based. Nonholonomic systems are constrained to a pattern of motion, such as a train on a track, or a rook in the game of chess: the train is confined to movement along its track, and the rook can only move in L-shaped patterns. In the case of a car, the system can be said to have nonholonomic constraints, in addition to being underactuated, since the car cannot physically move perpendicular to its forward direction of motion. For the purposes of the rest of this report, we shall refer to the underactuated non-holonomically constrained car system as simply the “nonholomic system.”

Since cars are nonholonomic systems and do not have a one to one correlation of control to position, they are much more challenging to control. To reiterate the parallel-parking example, to translate a car 10 inches to the left (increase y), the car must be driven forward, turned, and then reversed. The car cannot simply translate to the left, as can a fully-actuated spaceship or satellite with a number of rocket boosters.

An Intuitive Explanation for the Model

The basic motions of a car can be divided up into straight lines and semi-circles. When the steering wheel is straight ahead, the car goes straight with a speed according to the speed input, and when the steering wheel is turned, it follows the path of a circle proportional to the angle of the steering wheels. Therefore, the Cartesian coordinates of the path should be related to the sine and cosine of the steering angle, and the change in heading should be proportional to the steering angle too. For the rear-wheel drive vehicle, we followed the example in a paper published by the LAAS.⁵

The System Model

The kinematic state space for the plant model is defined below:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{q} \\ \dot{f} \end{bmatrix} = \begin{bmatrix} \cos q \\ \sin q \\ \tan q / \ell \\ 0 \end{bmatrix} v_1 + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} v_2$$

Equation 6: State space for plant (R/C car)

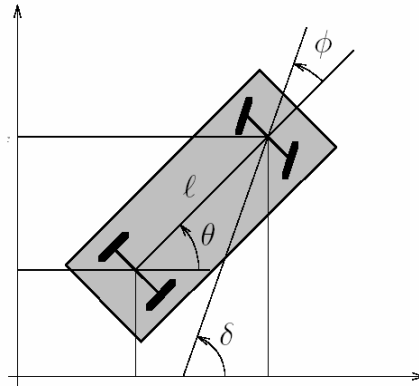


Figure 19: DeLuca, Oriolo, and Samson's diagram of the state space variables⁵

The x and y are the Cartesian coordinates of the position of the center of the rear axle, θ is the angle of the centerline of the car relative to the global x -axis, and ϕ is the steering angle (angle of the front wheels relative to the center line of the car), ℓ is a

constant equal the length of the wheelbase of the car, and v_1 and v_2 are the speed and steering inputs, respectively. See Figure 19.

With equations that describe the movement of the car based on control inputs, we applied the model to Simulink to see if it performed like the real thing. Figure 20 is the Simulink model.

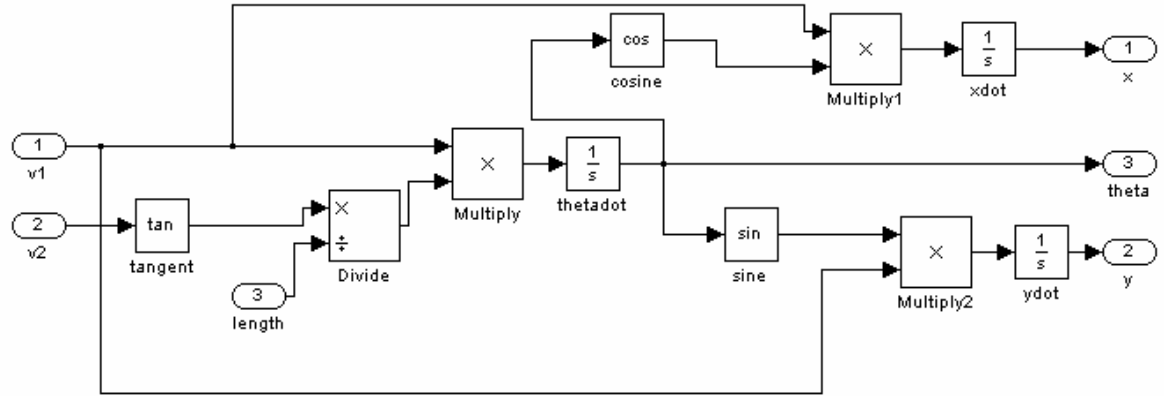


Figure 20: Simulink model of the plant (R/C car)

This model of the plant allows us to put in control inputs (v_1 and v_2) and see how the car will respond (x , y , θ , $\dot{f}$). To test the model, we input a speed step response and a steering step response. Since the steering input is constant, the car should travel in a circle of constant radius. Figure 21 shows the results of two, step inputs. The grid has dimensions of 640 by 480, since those are the dimensions of the pixels in the camera's vision. All speeds are represented in pixels/second. Starting the car in the middle of the grid, the throttle begins at $t=0$ with a value of 6 pixels/sec. The steering has a step response of $\pi/4$ after 6 seconds, and an additional $-\pi/2$ at 35 seconds. The result is a straight section followed by two circular arcs.

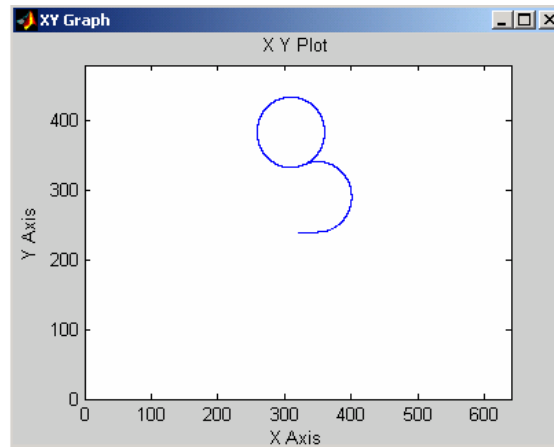


Figure 21: Simulation of speed and steering step response

Like all models, it is important to note that this model makes some assumptions and does not take into account a few important aspects of the real-life vehicle. First, this is a kinematic model and ignores any effects of motion of the vehicle. Most prominently, it ignores wheel slip, assuming that the car will turn the same radius circle regardless of its speed. Since the cars are geared down and operating at low speed, we feel that this assumption is reasonable. Second, the model assumes that the relation between control inputs and the actual actuator operation is one to one. However, although the steering and voltage responses turn out to be linear with respect to control input, both saturate at a maximum and minimum limit, which the model does not account for. Third, the model assumes no delay between the application of the control and the effect of the control. (e.g. – turning the front wheels will immediately cause the car to turn). We did not perform any experiments to determine if this was a significant issue, but we have had no problems that seem to stem from this limitation of the model.

The addition of dynamic effects into the model would make it more accurate for the motions of cars moving at variable, high speed and would allow us to design a controller to handle vehicles at these speeds. At the speeds of our vehicles, wheel slip is not an issue, but with faster cars, these issues would become more prominent and there would be significant gain from a dynamic car model.

Defining the Objective

The simplest way to model the system (plant and controller) is to use only one error variable for control: the perpendicular distance from the center of the car to the desired position (the method used in the ELE Car Lab project). This distance is defined as the error, and the controller for the car tries to minimize this error.

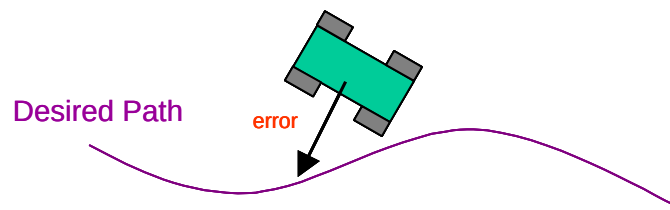


Figure 22: Diagram of a one error variable system

This model works well for simple systems, but it has obvious flaws. First, it does not take the orientation of the car into account; so therefore, it can never get a car to a desired position and orientation. Second, since there is no orientation control, the car may be at a position of zero distance error, but it will still have a large distance error when it tries to move forward. The primary flaw in the design is that without heading angle, the control inputs cannot be superimposed. The amount by which the error will be reduced by changing the steering is not consistent for all conditions of the car. Should the car be at a different orientation to the line, turning the wheel 15 degrees to the right will have a different effect on the future error, even if it has the same error distance. See Figure 23, where for the same distance error, the results of a step response will be significantly different depending on the heading of the vehicle.

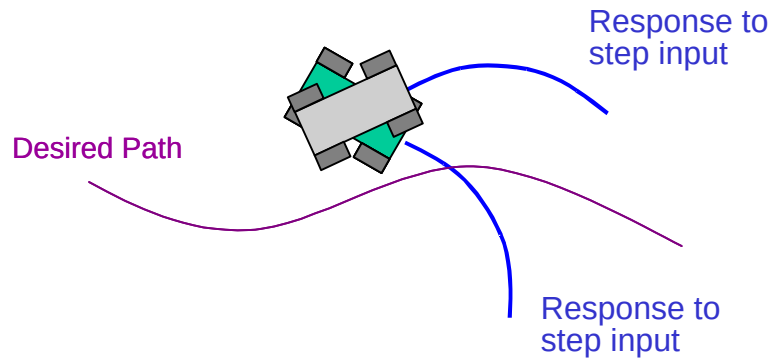


Figure 23: Inability to superimpose step responses with only distance error as the error variable

Therefore, to get a more accurate and stable controller, we will use information on the position, heading, and steering wheel angle as our state variables and effectively track both heading and distance errors. Distance error is calculated with the difference between the desired x, y and the current x, y , and heading error is the error in θ . Figure 24 shows the general control loop for the system and the types of inputs and outputs.

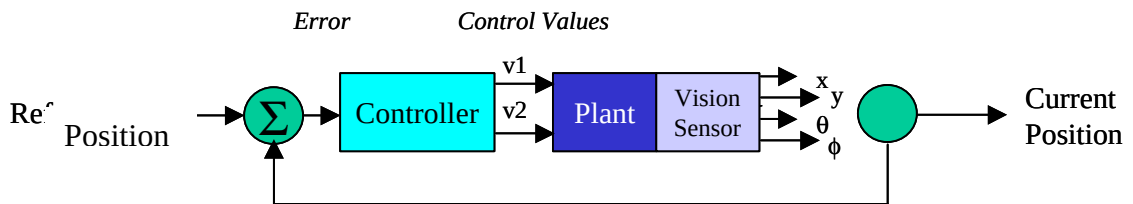


Figure 24: General control loop for the system

Since we now have four state variables (x, y, θ , and ϕ) and two output variables (steering and speed), the problem is no longer a single input-single output system and we will use modern control theory instead of classical control theory. Like the plant model, we built and tested the model in Simulink, then tried it out and compared it to the real system.

Modeling the Physical System

In order to make use of the car plant model, it must correlate to what is going on the real world. The actual remote controlled car needs inputs of voltages to the motor and to the

steering servo, rather than arbitrary v_1 and v_2 values. To the computer, these inputs are voltage values sent to the remote controllers, but they still must be translated from v_1 and v_2 , which are used by the control algorithm. In addition, while the virtual car is able to make tight turns and travel at any speed, the real car has limitations and non-linearities in its response. To correctly model the behavior of the actual car, we collected data on the performance of the car at numerous control settings for velocity and turning radius. For both properties, the response of the car is surprisingly linear, except at the limits, where it reaches a limit and saturates.

Velocity

The velocity response is linear throughout the low-speed area, and fairly symmetric forward and in reverse. When pushed beyond a certain limit in either direction, however, the speed reaches a maximum value and saturates

We then ran the car through multiple straight and curved paths and collected data for throttle voltage vs. speed, as listed in Figure 28. We then achieved a linear mapping between forward and backward input voltage and speed, v_1 (pixels/sec) using a linear regression as listed in Figure 29 and Figure 30.

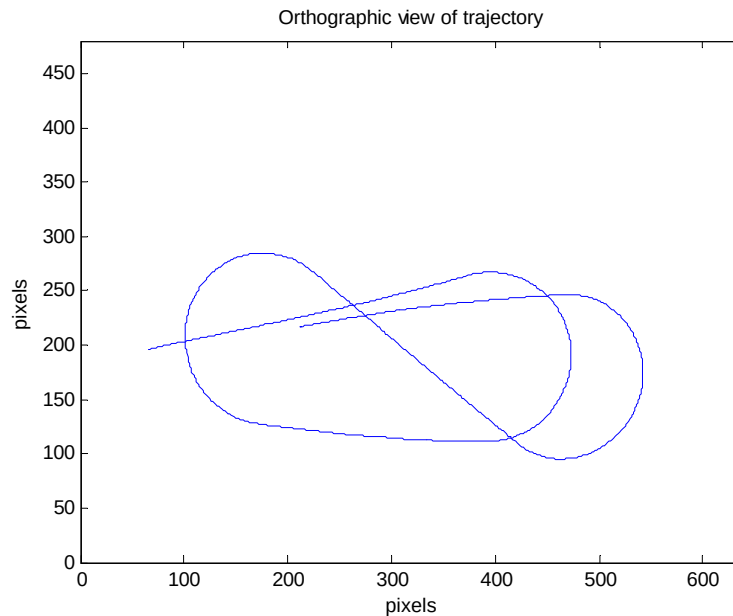


Figure 25: Orthographic view of trajectory

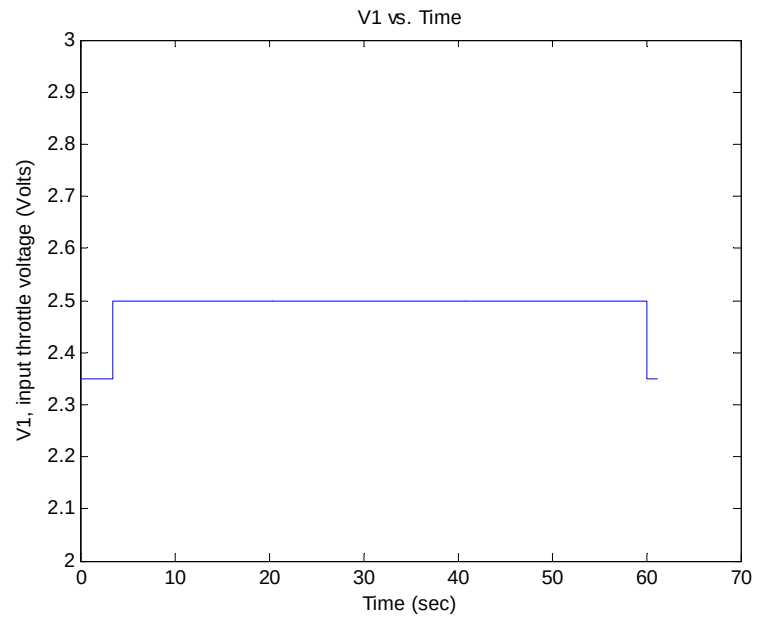


Figure 26: V1 vs. Time

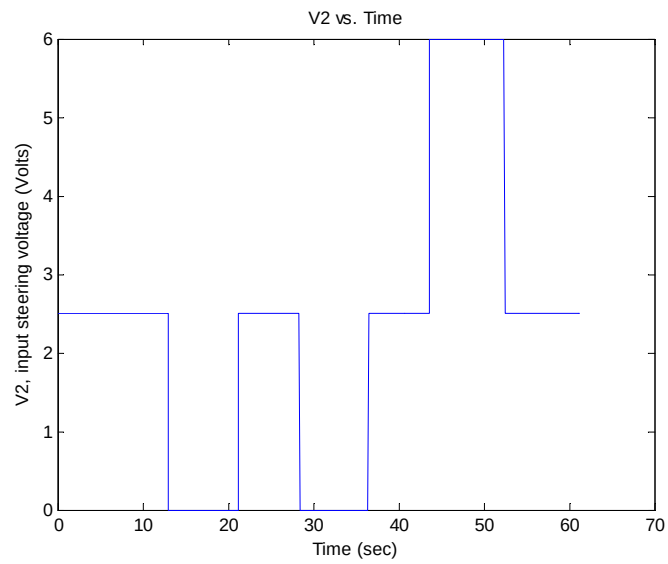


Figure 27: V2 vs. Time

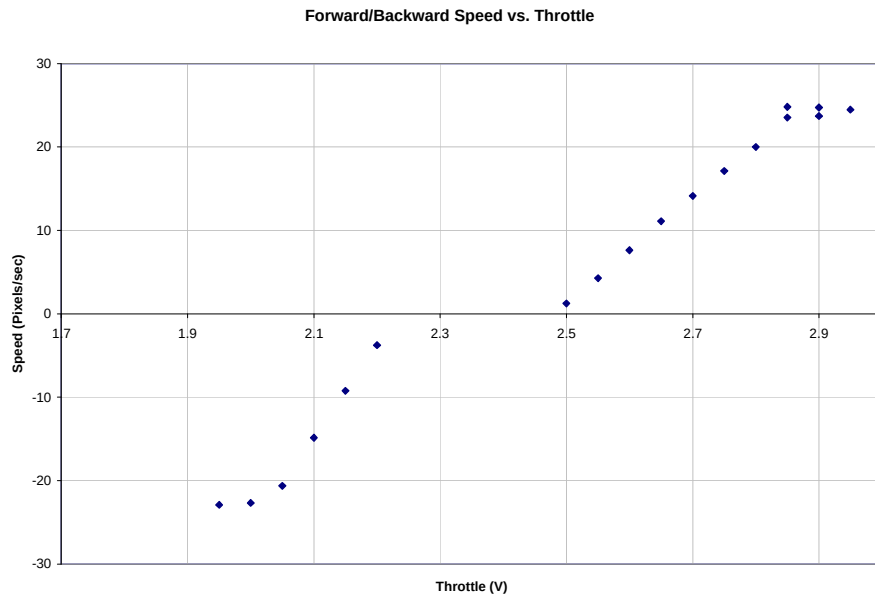


Figure 28: Speed vs. Throttle Voltage

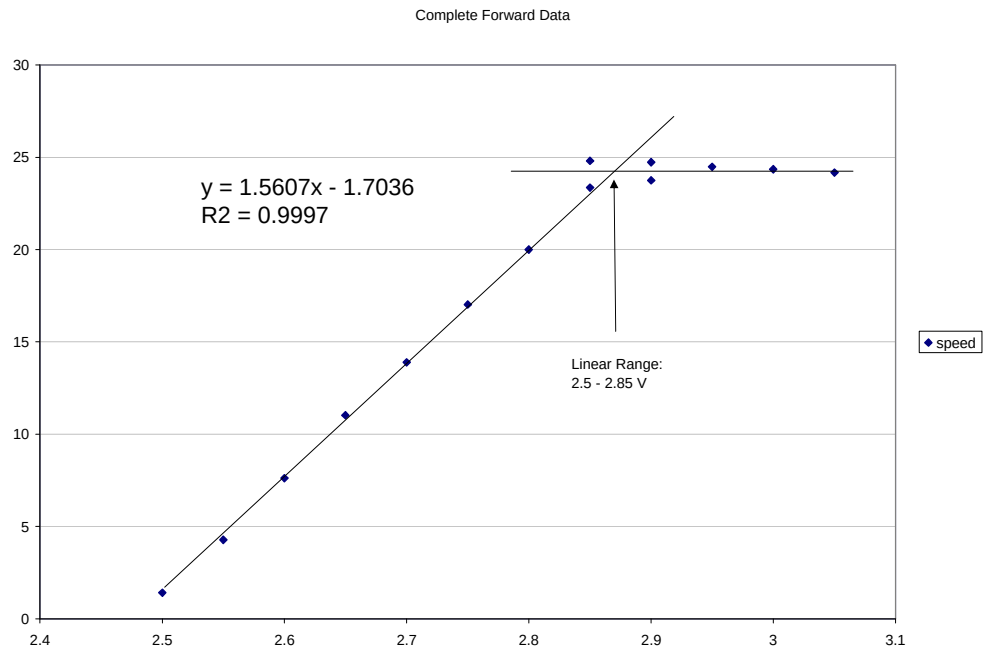


Figure 29: Forward speed vs. Throttle Voltage

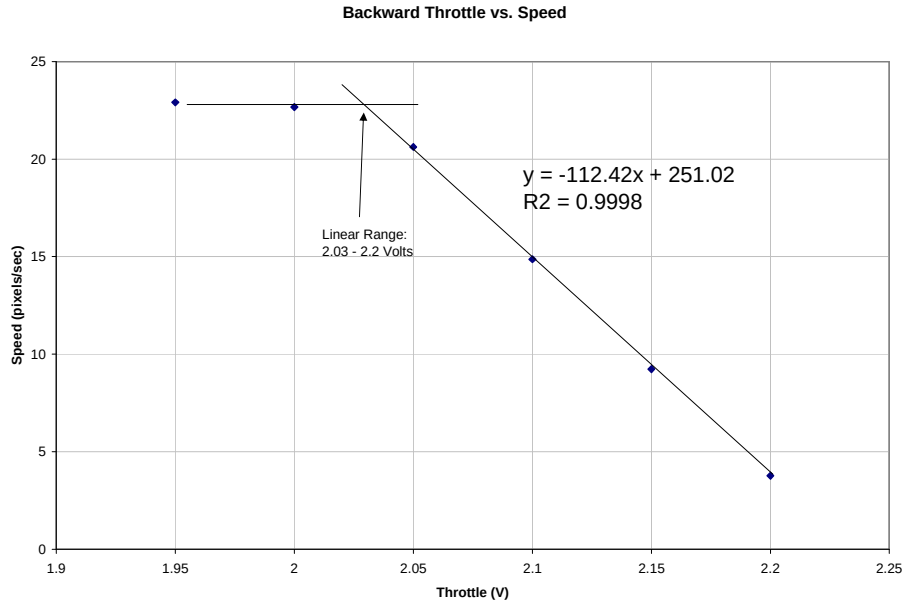


Figure 30: Backward Speed vs. Throttle Voltage

Steering

The steering response was somewhat more difficult to measure, since we used turning diameter as the dependent variable. However, capturing the location of the car via LabView proved to be very helpful. To calculate the conversion from voltage input to control input (v2) we needed to test the responses of both the real car and the car model and do a comparison. The common variable between the two systems was turning diameter, since it was easily measured on the real system using LabView, and on the car-model by graphing the simulation results. First, we set the steering input on the car to a fixed voltage, then drove it at slow speed (to eliminate slipping) and measured the turning diameter. See Figure 31 for a comparison of turning diameter and steering input voltage. The results are not exactly symmetric for right and left turns, but the disparity did not have a significant impact on the final system model.

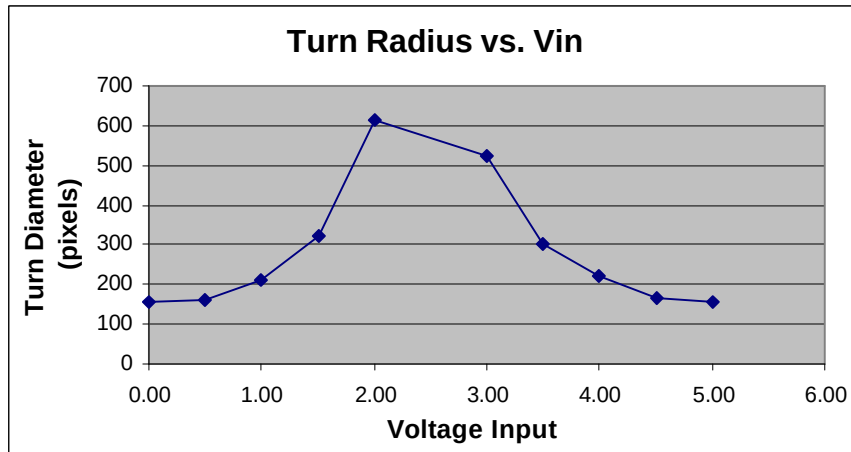


Figure 31: Turning Radius as a function of steering voltage

Next, we completed the same process, except with the car model, by setting a constant v_2 and measuring the resulting turning diameter. There was no need to test both right and left turns for the car model simulation, since the equations are symmetric. See Figure 32 for the relationship between v_2 and turn diameter.

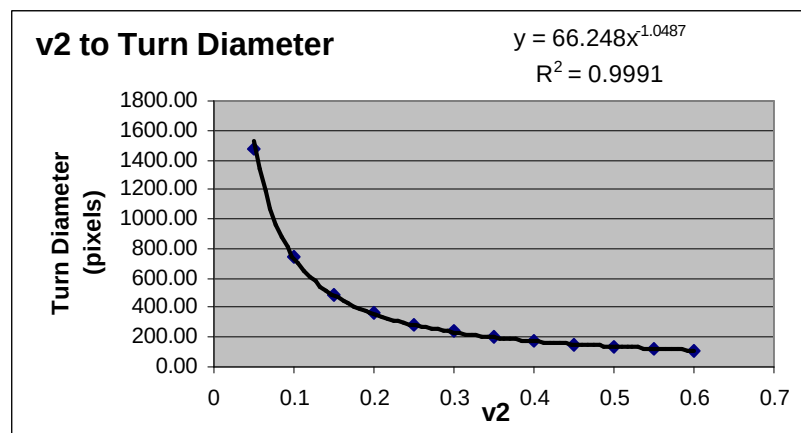


Figure 32: Turning Diameter as a function of v_2 (car model)

Now with a common variable for the two systems, we were able to compare them and find a relationship. Simply looking at the two graphs gives the appearance that they are both a kind of power function where the diameter goes to infinity at very small voltage or v_2 inputs, and goes to zero at very high voltage or v_2 inputs. So, although turning diameter varies approximately as a power function to voltage input, (since diameter goes to infinity as the car goes straight), the same occurs for the virtual car

model, and we found that the relationship between the steering voltage and the control input was very close to linear. See Figure 33.

Through our experiments, we also found that the steering of the car is asymmetric, with the car being able to turn a slightly tighter radius on one side than on the other. This difference accounts for the offset of the regression line. The voltage input for the wheels pointing straight ahead is 2.50 volts, while the regression line crosses at approximately 2.48 volts.

An important part of the conversion is the attention to limits, which differentiate the real car from the virtual car. The real car is limited to a wheel angle of about 35 degrees, which gives us a minimum turning diameter of about 160 pixels on our grid. To match the virtual car to the real car, any control values outside of this range are simply set to the maximum possible values (35 degrees or 160 pixel diameter), and the car turns a 160 pixel diameter turn. The final result is a very straight plot, with v2 varying linearly with voltage input, with limitations on voltage from about 0.5 volts to 4.5 volts.

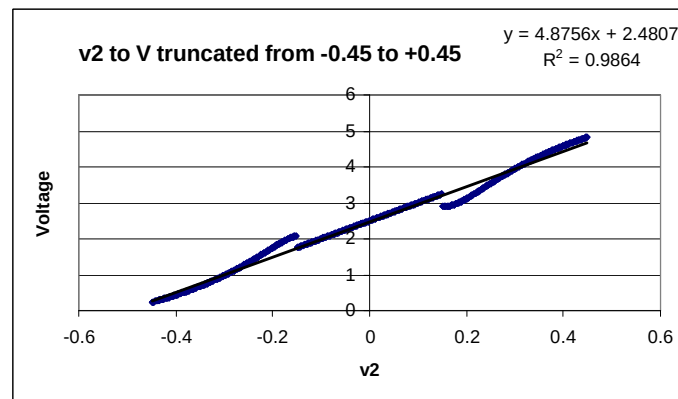


Figure 33: Control Inputs to Steering Voltage

With our data, we were also able to convert from V to v2, allowing us to simulate the action of the real car when put in the simulation.

Building the Controller

There were a variety of controller algorithms to choose from, each with its own advantages and disadvantages. The theoretical background and Simulink

implementations of each controller are listed in Appendix B. In addition, each method is rated in terms of functionality, complexity, performance, and intuitiveness in Table 1.

For our algorithm, we chose to implement the most intuitive algorithm, the “Linearized Original Coordinate Form,” as is elaborated in texts by Samson, et. al.^{6,7,8}

The state variables in this system are global x and y coordinates, θ , the angle of the car relative to the x -axis, and ϕ , the angle of the steering wheels relative to the car. With these variables, we will try to minimize the error in distance from the desired point (Δx , Δy), and minimize the heading error ($\Delta \theta$). ϕ is used to predict the current direction of the car and the extent to which the steering angle needs to be changed. Figure 34 illustrates this. There also exists an alternate algorithm that allows path convergence at any given velocity, i.e. without having to parameterize a reference path into a reference vehicle, and instead, having a vehicle drive with any arbitrary velocity and converge with a given path.⁹

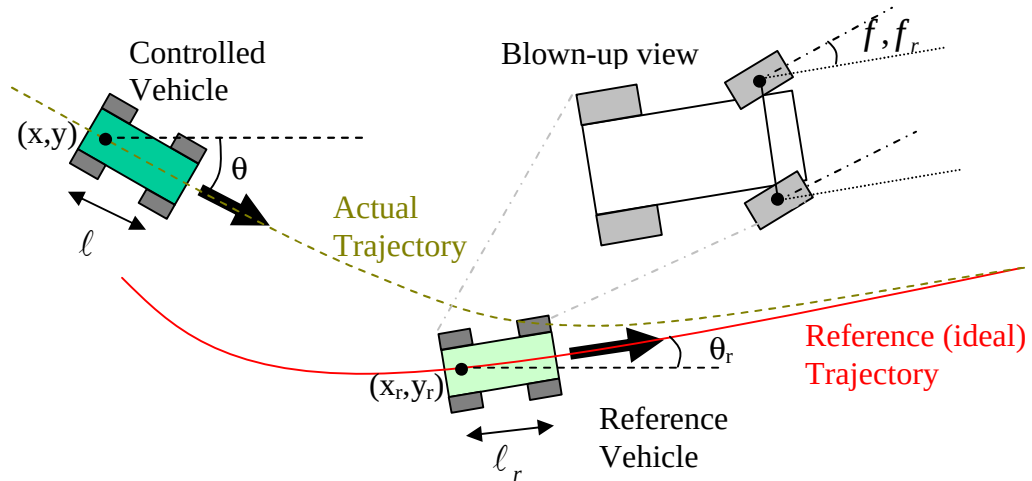


Figure 34: Rear Wheel Drive Kinematic Model Illustration

Approach	How it Works	Algorithmic Complexity	Performance	Is it Intuitive?
Linearized Chained Form	Maps coordinates to alternate chained form representation that conforms to canonical control theory, then linearizes closed loop system matrix, defines feedback constants such that closed loop eigenvalues have negative real part.	Memoryless, simple calculations done each iteration	Performs poorly on large initial error conditions, otherwise decent performance. Cannot track vertical lines, due to $q \in \left[-\frac{p}{2}, \frac{p}{2}\right]$	No. Mapping of coordinates into chained canonical form prevents an intuitive perturbing of feedback constants.
Linearized Original Coordinate Form	Rotates entire system into frame of reference vehicle, approximately linearizes error about reference trajectory, and defines feedback constants to conform with Lyapunov analysis, such that closed loop eigenvalues have negative real part.	Memoryless, simple calculations done each iteration, although more computation required than chained form	Performs poorly on large initial error convergence, otherwise decent performance in all reference trajectory domains.	Yes. Feedback constants control convergence of approximate perpendicular and parallel errors, in relation to direction of target vehicle.
Linearized Full-State Original Coordinate Form	Creates two extra auxiliary variables, and maps all state and control variables into a form where there are as many control variables as state variables.	Requires integration each iteration of two variables	Performs well on large and small initial errors; guarantees convergence to trajectory.	No. Most unintuitive of all solutions: mapping and addition of additional variables eliminates all intuition from feedback constants.

Table 1: Advantages and disadvantages for various control scenarios

First Approach: Approximate Linearization in Chained Form

Kinematic Equations

The following three algorithms and background theory are originally taken from the work of Claude Samson, Alessandro DeLuca, and Giuseppe Oriolo. [10], [11], [12], [13], [14], [15]. Table 1 describes the tradeoffs in using each algorithm. The model for the car we are using does not take into slip, drastically reducing the number of state variables involved. Slip, of course, can be taken into account, which merely predicts that the car will form looser turns as a function of forward velocity. For the scope of this project, however, we will assume that at low enough speeds, such dependence will not largely affect the steering curves. The model is defined as the following:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{q} \\ \dot{f} \end{bmatrix} = \begin{bmatrix} \cos q \\ \sin q \\ \tan f / \ell \\ 0 \end{bmatrix} v_1 + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} v_2$$

Equation 7: State Space for Plant (R/C Car)

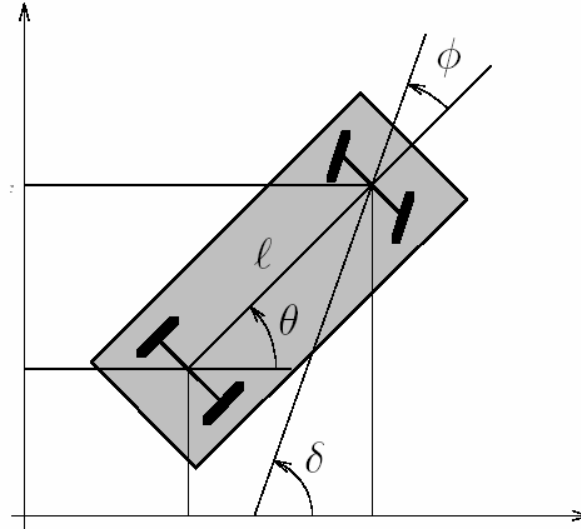


Figure 35: Car Model

Here we see that there are four state variables: x , y , q , and f , corresponding to the (x, y, q) absolute position of the vehicle & relative orientation, and a steering angle ϕ (where zero corresponds to no wheel turn). Above, ℓ corresponds to the length of the car. v_1 and v_2 correspond to forward throttle and steering input, respectively. In addition, an illustration of these variables in context can be found in Figure 34.

Chained Form

There are canonical forms for modern feedback and linearization for mobile wheeled robots. However, they operate especially efficiently and have cleaner algebra for state/control variables in the following form:

$$\begin{aligned}
\dot{x}_1 &= u_1 \\
\dot{x}_2 &= u_2 \\
\dot{x}_3 &= x_2 u_1 \\
&\vdots \\
\dot{x}_n &= x_{n-1} u_1
\end{aligned}$$

Equation 8: Chained Form Relationships

Equation 8 is called the (2, n) single-chain form. And it turns out that we can transform the state variables of Equation 7 into chained form using Equation 9.

$$\begin{aligned}
x_1 &= x \\
x_2 &= \frac{\tan f}{\ell \cos^3 q} \\
x_3 &= \tan q \\
x_4 &= y
\end{aligned}$$

$$\begin{aligned}
v_1 &= \frac{u_1}{\cos q} \\
v_2 &= \frac{-3 \sin q \sin^2 f u_1}{\ell \cos^2 q} + \ell \cos^3 q \cos^2 f u_2
\end{aligned}$$

Equation 9: Change of coordinates for controller

This transformation merely maps the 4 state variables and 2 control variables to another 4 state variables, $x_1 \dots x_4$, and two control variables, u_1, u_2 .

Linearization Control

Our current objective is to implement a control law to track a feasible trajectory and reject the effect of disturbances. Since our model involves four state variables and two control inputs, we must take all of these variables into account when calculating such things as sensor error. Not only do we need to take the difference between the desired (x, y) coordinate on the path, but we must also take the difference between a desired and current steering angle, and the difference between the desired and current orientation. Thus, even if we only have the (x, y) coordinates of the path, we need to retroactively calculate the rest of the state variables that would have been used to steer a car through

the desired path. To fully describe this “phantom” car, we need just as many state variables as our input car. We will denote the full desired state as the vectors $\mathbf{x}_d$ and $\mathbf{v}_d$. The subscript “d” indicates a ‘desired’ or reference value.

Even if we have a target path which does not have timing information coded into it (i.e. a set of points that define a circular path), we can use the following equations to retroactively calculate the rest of the state variables:

$$v_{d2}(t) = \ell v_{d1} \frac{(\ddot{y}_d \dot{x}_d - \ddot{x}_d \dot{y}_d) v_{d1}^2 - 3(\ddot{y}_d \dot{x}_d - \ddot{x}_d \dot{y}_d)(\dot{x}_d \ddot{x}_d + \dot{y}_d \ddot{y}_d)}{v_{d1}^6 + \ell^2 (\ddot{y}_d \dot{x}_d - \ddot{x}_d \dot{y}_d)^2}$$

Equation 10: Calculating phantom input from time-invariant path coordinates

where $(x_d(t), y_d(t))$ are the target path coordinates, $v_{d1}(t)$ can be a constant input velocity, and $v_{d2}(t)$ is the steering input that a phantom car would need to recreate the exact path. The phantom car’s state variables can then be mapped to chained form using Equation 9, yielding vectors $\mathbf{x}_d$, $\mathbf{u}_d$, for the desired state, and $\mathbf{x}$, $\mathbf{u}$ for the input car. Thus, we can now establish feedback equations. The following formulas are taken from general modern control theory involving trajectory tracking with nonholonomic forms¹⁶. The general system equations for the input car can be written as:

$$\dot{\mathbf{x}} = \mathbf{f}(t; \mathbf{x}) + \mathbf{B}(t)\mathbf{u} + \mathbf{D}(t)\mathbf{w}$$

Equation 11: Linearizing form

where $\mathbf{w}$ represents the disturbance input, and $\mathbf{f}(t, \mathbf{x})$ and $\mathbf{B}(t)$ are determined by Equation 8. For now, we will set $\mathbf{w} = 0$, ignoring the disturbance input. We are left with:

$$\dot{\mathbf{x}} = \mathbf{f}(t; \mathbf{x}) + \mathbf{B}(t)\mathbf{u}, \text{ with}$$

Equation 12: Linearizing form without disturbance

The desired state can be written in the same form:

$$\dot{\mathbf{x}}_d = \mathbf{f}(t; \mathbf{x}_d) + \mathbf{B}(t)\mathbf{u}_d$$

Equation 13: Linearizing form for desired state

Taking the difference of Equation 12 and Equation 13, we get:

$$\dot{\tilde{x}} = \tilde{f}(t; \tilde{x}) + B(t)\tilde{u}$$

Equation 14: Error Equation

where $\tilde{f}(t, \tilde{x}) = f(t, x) - f(t, x_d)$. We can then use a common trick where we approximately linearize the difference function, $\tilde{f}$ around the reference coordinates using the relationship:

$$A(t) = \left. \frac{\partial \tilde{f}(t; \tilde{x})}{\partial \tilde{x}} \right|_{\tilde{x}=0}, \tilde{B}(t) = B(t)$$

Equation 15: Computing matrices A(t) and B(t)

We now add and subtract $A(t)\tilde{x}$ to Equation 14, and substitute for $\tilde{B}(t)$, yielding:

$$\dot{\tilde{x}} = A(t)\tilde{x} + B(t)\tilde{u} + [\tilde{f}(t; \tilde{x}) - A(t)\tilde{x}]$$

The term in square brackets is the only remaining non-linear term, and is $o(|\tilde{x}|)$,

so

$$\lim_{x_e \rightarrow 0} \left(\frac{o(|x_e|)}{|x_e|} \right) = 0$$

And hence, we can ignore the term in square brackets. Thus, we are left with a completely linearized system:

$$\dot{\tilde{x}} = A(t)\tilde{x} + B(t)\tilde{u}$$

where $A(t)$ and $B(t)$ come from the approximate linearization about $\tilde{x}$ and $\tilde{u}$ using Equation 15. Hence, for our chained-form case of the underactuated vehicle, we are left with:

$$\tilde{\dot{x}}_1 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & u_{d1}(t) & 0 & 0 \\ 0 & 0 & u_{d1}(t) & 0 \end{bmatrix} \tilde{x} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ x_{d2}(t) & 0 \\ x_{d3}(t) & 0 \end{bmatrix} \tilde{u}$$

Equation 16: Linear time-varying error function for controller

Now, we arbitrarily designate the vector $\tilde{u}$ as the following:

$$\begin{aligned} \tilde{u}_1 &= -k_1 \tilde{x}_1 \\ \tilde{u}_2 &= -k_2 \tilde{x}_2 - \frac{k_3}{u_{d1}} \tilde{x}_3 - \frac{k_4}{u_{d1}^2} \tilde{x}_4 \end{aligned}$$

Equation 17: Design of Control Variables

yielding:

$$A_{cl}(t) = \begin{bmatrix} -k_1 & 0 & 0 & 0 \\ 0 & -k_2 & -k_3/u_{d1}(t) & -k_4/u_{d1}^2(t) \\ -k_1 x_{d2}(t) u_{d1}(t) & u_{d1}(t) & 0 & 0 \\ -k_1 x_{d3}(t) & 0 & u_{d1}(t) & 0 \end{bmatrix}$$

Equation 18: Closed Loop System Matrix

At first, this choice for the control input may appear to be wholly arbitrary. However, upon choosing the control inputs for the target vehicle as in Equation 17, this closed-loop system matrix will yield an error that converges to zero, since it has constant eigenvalues with negative real part. Such a constraint on the matrix is fulfilled only if the following polynomial:

$$l^3 + k_2 l^2 + k_3 l + k_4$$

Equation 19: Hurwitz Stability Polynomial

is a Hurwitz-Routh polynomial, or in other words, a polynomial with real positive coefficients and roots which are either negative or pair wise conjugate with negative real parts. This is an analogous idea to having a closed-loop system gain polynomial have poles in the left-hand plane in order for the system to be asymptotically stable. Thus, the

overall control input is $u = u_d + \tilde{u}$, or the sum of the feed forward and feedback component. If the desired state is reached, then $\tilde{u} = 0$, and the control input, u simply reduces to the feed forward inputs u_d , and hence, the car will follow the desired path exactly.

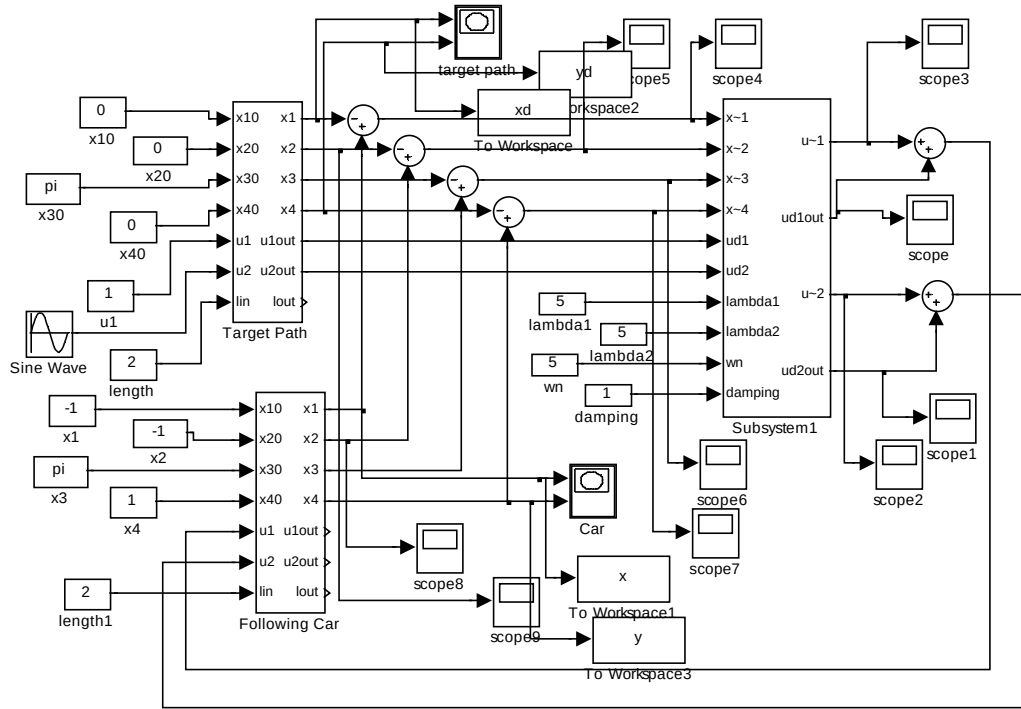


Figure 36: Simulink realization of Linearized Chained-form Controller

Thus, we have here our most important set of equations: the classical feedback equations for underactuated robot-like cars. Using these equations above, Equation 17 and Equation 19, we implemented the equations into Simulink as shown in Figure 36. We then ran simulations, having a controlled vehicle converge to a sample reference sinusoidal trajectory, which is shown in Figure 37, for four different starting coordinates. The steady-state error convergence for the state variables and control input is shown in Figure 38, with a sinusoidal reference trajectory. It should be noted, however, due to Equation 17, the system is not able to reference trajectories where $u_{d1} = 0$, i.e.

$\cos(q) \neq \left[-\frac{p}{2}, \frac{p}{2}\right]$. Thus, the algorithm is only suited for simulations where the orientation is never completely vertical, i.e., for only the open and connected first and fourth quadrants in q . However, as can be seen in Figure 37, the algorithm is extremely “smart,” in that it can even tell the car to back up to converge to the target trajectory, as demonstrated by the green and red paths.

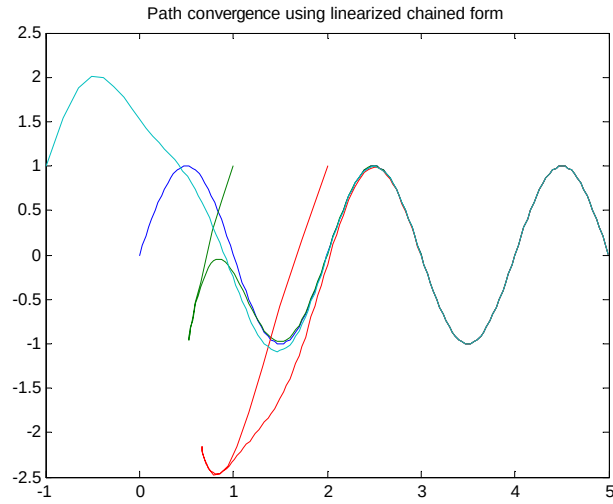


Figure 37: Simulink output for Chained Form Controller

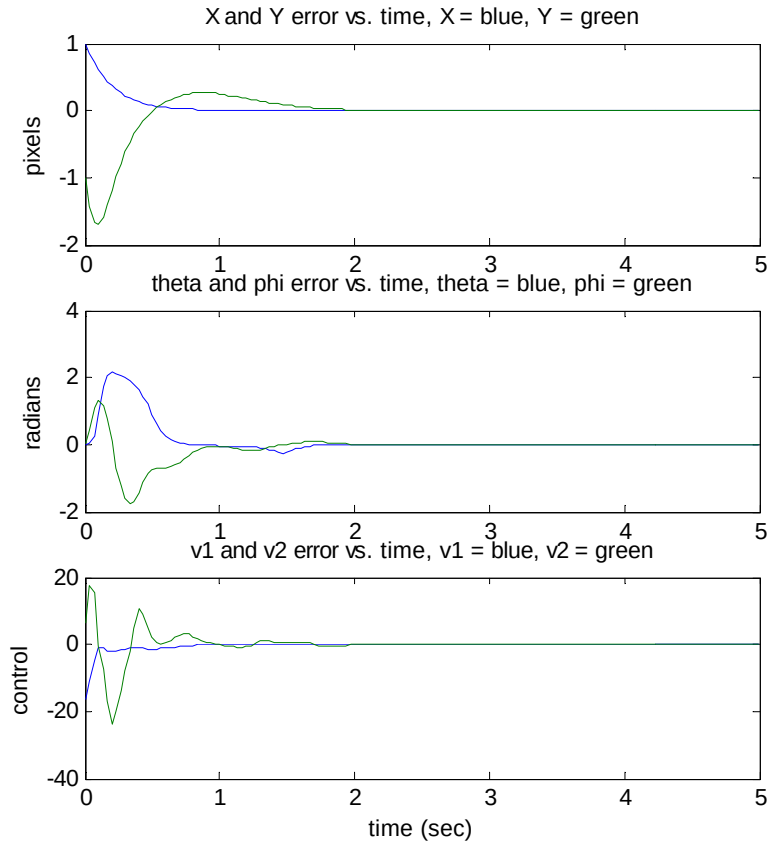


Figure 38: Errors vs. Time for Chained-Form Linearized Controller

Second Approach: Approximately Linearized System in Original Coordinates

In order to solve the problem of the discontinuities of the domain of q , we use the same methods as the approximately linearized chained form, except without mapping the original system into chained form. At the expense of using a non-canonical approach to the problem, we achieve a controller that can stabilize a car to any reference trajectory, such that the domain of q belongs to all open connected quadrants. In addition, this formulation allows the constants k_1 , k_2 , k_3 , and k_4 to be manipulated intuitively, since this method results in two decoupled classical control systems, one for path-oriented error, and another for perpendicular-to-path-oriented error.

Model Description

The variables are denoted similarly, according to **Figure 34**, as the following.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{q} \\ \dot{f} \end{bmatrix} = \begin{bmatrix} \cos q \\ \sin q \\ \tan f / \ell \\ 0 \end{bmatrix} v_1 + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} v_2 ,$$

$$\begin{bmatrix} \dot{x}_{ref} \\ \dot{y}_{ref} \\ \dot{q}_{ref} \\ \dot{f}_{ref} \end{bmatrix} = \begin{bmatrix} \cos q_{ref} \\ \sin q_{ref} \\ \tan f_{ref} / \ell_{ref} \\ 0 \end{bmatrix} v_{1,ref} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} v_{2,ref}$$

Equation 20: Kinematic Model for Control and Reference Vehicles

First, we assume that $\ell_{ref} = \ell$ (i.e. the two cars are of the same length). We then define the error between the state variables:

$$\begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{q} \\ \tilde{f} \end{pmatrix} = \begin{pmatrix} x - x_{ref} \\ y - y_{ref} \\ q - q_{ref} \\ f - f_{ref} \end{pmatrix}$$

We then apply a rotation, using the heading of the reference vehicle, q_{ref} , as the rotation angle.

$$\begin{pmatrix} \tilde{x}_{rot} \\ \tilde{y}_{rot} \\ \tilde{q}_{rot} \\ \tilde{f}_{rot} \end{pmatrix} = \begin{pmatrix} \cos q_{ref} & \sin q_{ref} & 0 & 0 \\ -\sin q_{ref} & \cos q_{ref} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{q} \\ \tilde{f} \end{pmatrix}$$

Thus, the error variable, $\tilde{x}_{rot}$ corresponds to the position error along the reference path and $\tilde{y}_{rot}$ corresponds to the position error perpendicular to the reference path. We then take the derivative of the error variables with respect to time:

$$\begin{pmatrix} \dot{\tilde{x}}_{rot} \\ \dot{\tilde{y}}_{rot} \\ \dot{\tilde{q}}_{rot} \\ \dot{\tilde{f}}_{rot} \end{pmatrix} = \dot{\mathbf{q}}_{ref} \begin{pmatrix} -\sin q_{ref} & \cos q_{ref} & 0 & 0 \\ -\cos q_{ref} & -\sin q_{ref} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{q} \\ \tilde{f} \end{pmatrix} + \begin{pmatrix} \cos q_{ref} & \sin q_{ref} & 0 & 0 \\ -\sin q_{ref} & \cos q_{ref} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \dot{\tilde{x}} \\ \dot{\tilde{y}} \\ \dot{\tilde{q}} \\ \dot{\tilde{f}} \end{pmatrix}$$

yielding:

$$\begin{pmatrix} \dot{\tilde{x}}_{rot} \\ \dot{\tilde{y}}_{rot} \\ \dot{\tilde{q}}_{rot} \\ \dot{\tilde{f}}_{rot} \end{pmatrix} = \begin{pmatrix} \dot{\mathbf{q}}_{ref} (-\tilde{x} \sin q_{ref} + \tilde{y} \cos q_{ref}) + \dot{\tilde{x}} \cos q_{ref} + \dot{\tilde{y}} \sin q_{ref} \\ \dot{\mathbf{q}}_{ref} (-\tilde{x} \cos q_{ref} - \tilde{y} \sin q_{ref}) - \dot{\tilde{x}} \sin q_{ref} + \dot{\tilde{y}} \cos q_{ref} \\ \dot{\tilde{q}} \\ \dot{\tilde{f}} \end{pmatrix}$$

Now, making the following substitutions:

$$\dot{\mathbf{q}}_{ref} = \frac{v_{1,ref}}{\ell} \tan f_{ref},$$

$$-\tilde{x} \sin q_{ref} + \tilde{y} \cos q_{ref} = \tilde{y}_{rot},$$

$$-\tilde{x} \cos q_{ref} - \tilde{y} \sin q_{ref} = -\tilde{x}_{rot},$$

$$\dot{\tilde{x}} \cos q_{ref} + \dot{\tilde{y}} \sin q_{ref} = (v_1 \cos q - v_{1,ref} \cos q_{ref}) \cos q_{ref} + (v_1 \sin q - v_{1,ref} \sin q_{ref}) \sin q_{ref}$$

$$= v_1 (\cos q \cos q_{ref} + \sin q \sin q_{ref}) - (v_{1,ref} \cos^2 q_{ref} + v_{1,ref} \sin^2 q_{ref})$$

$$= v_1 (\cos(q - q_{ref})) - v_{1,ref} = v_1 \cos \tilde{q} - v_{1,ref},$$

$$-\dot{\tilde{x}} \sin q_{ref} + \dot{\tilde{y}} \cos q_{ref} = -(v_1 \cos q - v_{1,ref} \cos q_{ref}) \sin q_{ref} + (v_1 \sin q - v_{1,ref} \sin q_{ref}) \cos q_{ref}$$

$$= v_1 (\sin(q - q_{ref})) = v_1 \sin \tilde{q},$$

$$\dot{\tilde{q}} = \dot{q} - \dot{\mathbf{q}}_{ref} = \frac{v_1}{\ell} \tan f - \frac{v_{1,ref}}{\ell} \tan f_{ref},$$

$$\dot{\tilde{f}} = \dot{f} - \dot{f}_{ref} = v_2 - v_{2,ref},$$

we are left with:

$$\begin{pmatrix} \dot{\tilde{x}}_{rot} \\ \dot{\tilde{y}}_{rot} \\ \dot{\tilde{q}}_{rot} \\ \dot{\tilde{f}}_{rot} \end{pmatrix} = \begin{pmatrix} \frac{v_{1,ref}}{\ell} \tan f_{ref} \tilde{y}_{rot} + v_1 \cos \tilde{q} - v_{1,ref} \\ -\frac{v_{1,ref}}{\ell} \tan f \tilde{x}_{rot} + v_1 \sin \tilde{q} \\ \frac{v_1}{\ell} \tan f - \frac{v_{1,ref}}{\ell} \tan f_{ref} \\ v_2 - v_{2,ref} \end{pmatrix}$$

In order to further proceed, we require another change of variables. Let us define a new mapping for our error, z:

$$\begin{pmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix} = \begin{pmatrix} \tilde{x}_{rot} \\ \tilde{y}_{rot} \\ \tan \tilde{q}_{rot} \\ \frac{\tan f - \cos \tilde{q} \tan f_{ref}}{\ell \cos^3 \tilde{q}} + k_2 \tilde{y}_{rot} \end{pmatrix}$$

and

$$w_1 = v_1 \cos \tilde{q} - v_{1,ref}$$

$$w_2 = \dot{z}_4 = k_2 \dot{\tilde{y}}_{rot} + \left(\frac{3 \tan f}{\cos \tilde{q}_{rot}} - 2 \tan f_{ref} \right) \frac{\sin \tilde{q}_{rot}}{\ell \cos^3 \tilde{q}_{rot}} \dot{\tilde{q}}_{rot} - \frac{v_{2,ref}}{\ell \cos^2 f_{ref} \cos^2 \tilde{q}_{rot}} + \frac{v_2}{\ell \cos^2 f \cos^2 \tilde{q}_{rot}}$$

Equation 21: Settings for w1, w2

Thus, we take the derivative of our new state variables, z, and we get, in terms of our new mapped variables:

$$\begin{pmatrix} \dot{z}_1 \\ \dot{z}_2 \\ \dot{z}_3 \\ \dot{z}_4 \end{pmatrix} = \begin{pmatrix} v_{2,ref} z_2 + w_1 \\ -v_{2,ref} z_1 + v_{1,ref} z_3 + w_1 z_3 \\ -k_2 v_{1,ref} z_2 + v_{1,ref} z_4 + w_1 \left(z_4 - k_2 z_2 + (1 + z_3^2) \frac{\tan f_{ref}}{\ell} \right) \\ w_2 \end{pmatrix}$$

Equation 22: Error Equations

Now, let us be reminded of our objective here: we wish to find a suitable value of v_1 and v_2 , the original control variables. v_1 and v_2 are both ‘encoded’ into w_1 and w_2 . Therefore, we must first find a setting for w_1 and w_2 that guarantees stability for Equation 22 above.

Thus, in order to guarantee stability for the error equations above, we perform a Lyapunov stability analysis. A ‘Lyapunov candidate function’ is a scalar function that is a measure of the amount of total error in a feedback system. For our particular system, it will be written as $V(z_1, z_2, z_3, z_4)$, i.e. a function of the error variables z_1, z_2, z_3, z_4 . In order to guarantee stability of a feedback system, the Lyapunov candidate function must meet three criteria:

- 1.) It must be positive definite over all domains of the input variables.
- 2.) It must have continuous partial derivatives over its domain, D .
- 3.) Its first derivative must be less than or equal to zero over its domain, D .

Here is an intuitive explanation for these three criteria: we wish to characterize the aggregate system error as a positive function where the error eventually goes to zero over time. Thus, the time-derivative of V must be negative over the entire domain, D , so that the value V will always be decreasing over time, with the absolute minimum value of V set to zero. Also, V must be a convex function over its domain, i.e. we must guarantee that all local minima are global minima. Such a requirement is satisfied only if the first derivative is always negative over the entire domain D .

Thus, a candidate Lyapunov function for our error variables, z_1, z_2, z_3, z_4 , is:

$$V(z_1, z_2, z_3, z_4) = \frac{1}{2} \left(z_1^2 + z_2^2 + \frac{z_3^2}{k_2} + \frac{z_4^2}{k_2 k_3} \right)$$

We can verify that V satisfies the first two criteria: it is positive definite and has continuous partial derivatives. To verify the third criteria, let us now differentiate V with respect to time:

$$\dot{V} = z_1 \dot{z}_1 + z_2 \dot{z}_2 + \frac{z_3 \dot{z}_3}{k_2} + \frac{z_4 \dot{z}_4}{k_2 k_3},$$

becomes:

$$\dot{V} = w_1 \left(z_1 + \frac{z_3}{k_2} \left(z_4 + (1 + z_3^2) \frac{\tan f_r}{\ell} \right) \right) + \frac{z_4}{k_2 k_3} (w_2 + v_{1,ref} z_3 k_3)$$

Thus, we can render the above equation negative over all domains of z if we set can designate w_1 and w_2 (which are mappings of v_1 and v_2) as:

$$w_1 = -k_1 |v_{1,ref}| \left(z_1 + \frac{z_3}{k_2} \left(z_4 + (1 + z_3^2) \frac{\tan f_r}{\ell} \right) \right)$$

and

$$w_2 = -k_3 v_{1,ref} z_3 - k_4 |v_{1,ref}| z_4$$

Equation 23: w_1 and w_2 optimal settings for Lyapunov function

Using the values for w , we attain:

$$\dot{V} = -k_1 |v_{1,ref}| \left(z_1 + \frac{z_3}{k_2} \left(z_4 + (1 + z_3^2) \frac{\tan f_r}{\ell} \right) \right)^2 + \frac{-k_4 |v_{1,ref}| z_4^2}{k_2 k_3}$$

which is negative for all values of z , provided that the constants $k_{1,2,3,4}$ are all positive.

Thus, combining Equation 21 and Equation 23, and solving for v_1 and v_2 , we get out input commands:

$$v_1 = \frac{v_{1,ref} - k_1 |v_{1,ref}| \left(z_1 + \frac{z_3}{k_2} \left(z_4 + (1 + z_3^2) \frac{\tan f_r}{\ell} \right) \right)}{\cos \tilde{q}_{rot}}$$

$$v_2 = \ell \cos^2 f \cos^2 \tilde{q}_{rot} \left(-k_3 v_{1,ref} z_3 - k_4 |v_{1,ref}| z_4 - \left(k_2 \dot{y}_{rot} + \left(\frac{3 \tan f}{\cos \tilde{q}_{rot}} - 2 \tan f_{ref} \right) \frac{\sin \tilde{q}_{rot}}{\ell \cos^3 \tilde{q}_{rot}} \dot{\tilde{q}}_{rot} - \frac{v_{2,ref}}{\ell \cos^2 f_{ref} \cos^2 \tilde{q}_{rot}} \right) \right)$$

Equation 24: Settings for v_1, v_2

making the substitutions,

$$\begin{aligned}\dot{\tilde{y}}_{rot} &= -\frac{v_{1,ref}}{\ell} \tan f \tilde{x}_{rot} + v_1 \sin \tilde{q} \\ \dot{\tilde{q}}_{rot} &= \frac{v_1}{\ell} \tan f - \frac{v_{1,ref}}{\ell} \tan f_{ref}\end{aligned}$$

We are left with:

$$\begin{aligned}v_1 &= \frac{v_{1,ref} - k_1 |v_{1,ref}| \left(z_1 + \frac{z_3}{k_2} \left(z_4 + (1+z_3^2) \frac{\tan f_r}{\ell} \right) \right)}{\cos \tilde{q}_{\alpha}} \\ v_2 &= \ell \cos^2 f \cos^2 \tilde{q}_{\alpha} \left(-k_3 v_{1,ref} z_3 - k_4 |v_{1,ref}| z_4 - \left(k_2 \left(-\frac{v_{1,ref}}{\ell} \tan f \tilde{x}_{\alpha} + v_1 \sin \tilde{q} \right) + \left(\frac{3 \tan f}{\cos \tilde{q}_{\alpha}} - 2 \tan f_{ref} \right) \frac{\sin \tilde{q}_{\alpha}}{\ell \cos^3 \tilde{q}_{\alpha}} \left(\frac{v_1}{\ell} \tan f - \frac{v_{1,ref}}{\ell} \tan f_{ref} \right) - \frac{v_{2,ref}}{\ell \cos^2 f_{ref} \cos^2 \tilde{q}_{\alpha}} \right) \right)\end{aligned}$$

Equation 25: Settings for v1, v2 to guarantee convergence

with $k_{1,2,3,4}$ positive. Using these inputs, we guarantee asymptotic convergence to zero error for the linearized system.¹⁷

However, how do we select the gains k to ensure “good” performance?¹⁸ First, we linearize the error system, Equation 22, and set $v_{2,ref} = 0$, i.e. for the case where the car is tracking a straight line. Intuitively this makes sense for the following two reasons:

- 1.) If v_1 , the throttle, is set to zero, then no matter what v_2 , the steering, does, the system will never converge to the target reference vehicle; the vehicle will just stand in place. Thus, v_1 , and thus, $v_{1,ref}$, must be nonzero for convergence.
- 2.) After rotating the errors through q_{ref} , the objective effectively drops its dependence on the steering input v_2 . This so happens because during each iteration the problem is reformulated into an on-trajectory and perpendicular error problem, regardless of the car’s orientation or steering input.

Thus, after linearizing and setting $v_{2,ref} = 0$, we get:

$$\begin{pmatrix} \dot{z}_1 \\ \dot{z}_2 \\ \dot{z}_3 \\ \dot{z}_4 \end{pmatrix} = \begin{pmatrix} w_1 \\ v_{1,ref} z_3 \\ -k_2 v_{1,ref} z_2 + v_{1,ref} z_4 \\ w_2 \end{pmatrix}$$

Note that this system is actually two decoupled systems:

$$\dot{z}_1 = w_1$$

and

$$\begin{pmatrix} \dot{z}_2 \\ \dot{z}_3 \\ \dot{z}_4 \end{pmatrix} = \begin{pmatrix} v_{1,ref} z_3 \\ -k_2 v_{1,ref} z_2 + v_{1,ref} z_4 \\ w_2 \end{pmatrix}$$

These difference equations are not linear, however – the last step is to define a new time-scaled set of inputs, p_1 , and p_2 . In addition, we also let:

$$t = s(t)$$

where

$$s(t) = \int_0^t |v_{1,ref}| dt$$

thus,

$$\dot{s}(t) = |v_{1,ref}|$$

Hence, in this newly-defined time system denoted by z' , we have:

$$\dot{z}' \dot{s}(t) = \dot{z}$$

thus,

$$\dot{z}' = \frac{\dot{z}}{\dot{s}(t)} = \frac{\dot{z}}{|v_{1,ref}|}$$

Let:

$$w_1 = |v_{1,ref}| p_1$$

$$w_2 = |v_{1,ref}| p_2$$

Thus, we are left with the following linearized, time-invariant error system:

$$\dot{z}'_1 = p_1$$

and

$$\begin{pmatrix} \dot{z}_2' \\ \dot{z}_3' \\ \dot{z}_4' \end{pmatrix} = \begin{pmatrix} \text{sign}(v_{1,ref}) z_3' \\ -k_2 \text{sign}(v_{1,ref}) z_2' + \text{sign}(v_{1,ref}) z_4' \\ p_2 \end{pmatrix}$$

Equation 26: Fully Linearized, time-invariant time-scaled error system

Thus, we can then set p_1 and p_2 as the following and choose the gains k such that the closed-loop poles lie in the left-hand plane:

$$\begin{aligned} p_1 &= -k_1 z_1' \\ p_2 &= -k_3 \text{sign}(v_{1ref}) z_3' - k_4 z_4' \end{aligned}$$

Equation 27: Gains as determined by time-scaled system

Consequently, these settings for the constants $k_{1,2,3,4}$, although not directly guaranteed through Lyapunov analysis to be stable, are well-suited for transient error performance for this particular straight-line case. Hence, they can be substituted into Equation 25.

Application of the Algorithm

There still are two issues left, however, before we implement this algorithm.

- 1.) The control value v_2 is actually the derivative of f . The R/C cars work by sending a voltage through a DAC channel which controls f directly. Thus, we need to first integrate v_2 with respect to time to get f , before we send the mapped f to a steering servo voltage.
- 4.) The constants, although ideal for operation in Simulink do not take into account the physical model limitations, such as neither maximum steering angle nor delay time in steering control input.

Thus, although we have a good set of inputs as described by Equation 25 and gains described by Equation 27, we still needed to tweak the simulated result so that it had good transient error performance.

For the actual implementation, we assumed the following, for smooth path following:

- 1.) The deviation from the mean is close enough to zero so that v_{1ref} and v_{2ref} are small.
- 2.) Also, $\tilde{q}_{rot}$ is also small, so $\cos\tilde{q}_{rot} = 1$.

We can throw out any nonlinear terms, taking the first order Taylor approximation of all control expressions. Thus, we reduce

Equation 24 to:

$$\begin{aligned} v_1 &= -k_1 |v_{1ref}| z_1 \\ v_2 &= -k_3 v_{1ref} z_3 - k_4 |v_{1ref}| z_4 \end{aligned}$$

Equation 28: Reduced Complexity settings for v1, v2

The feedback gains were then chosen accordingly first roughly to satisfy our Simulink model, and then were tweaked in Lab View further.

Finding the Gain Values k

The control data, listed in

Figure 27, was fed to the car running at a throttle voltage of 2.5V. The resulting curve is shown in Figure 25. In order to accurately model the impulse response of the steering, and to ensure the creation of a controller that does not exceed the maximum bounds of the control variable, v_2 , we first isolate the first step-response of the throttle input at time = 13 seconds, rotate the sample data so that the initial trajectory is parallel with the x-axis, and we track the deviation along the y-axis as a function of time. In accordance with the theory above, we rotate the world coordinate frame each iteration into that of the reference vehicle. Also, since the control gains do not depend on v_2 , the steering input,

we can set the “reference vehicle’s trajectory to a straight line and track the error deviation perpendicular to that straight line.

We then use a rotation matrix to rotate the (x, y) trajectory, based on the initial heading of the car, $q(t = 0)$. The first curve is then isolated, and the step response is graphed orthographically, as in Figure 40, and for y-error as a function of time, as in Figure 41. The transfer gain, from y to steering angle, f , was modeled as accurately as possible by the following equation:

$$\frac{Y_{err}(S)}{f_{in}(S)} = \frac{32.35}{s^2 + 0.3}$$

The resulting step response is plotted versus the actual model data in Figure 42. Hence, a value of $k_2 = 0.3$ was found to be the limiting constant on the steering feedback term, with an overall y-to-theta gain of 32.35. The rest of the constants were found by running the Lab View line-tracking algorithm repeatedly and testing which constants would provide smooth performance.

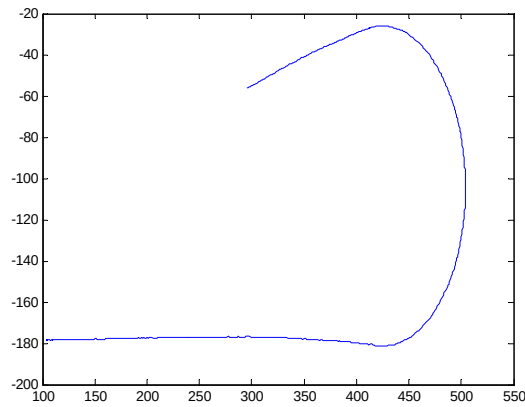


Figure 39: Rotated first turn

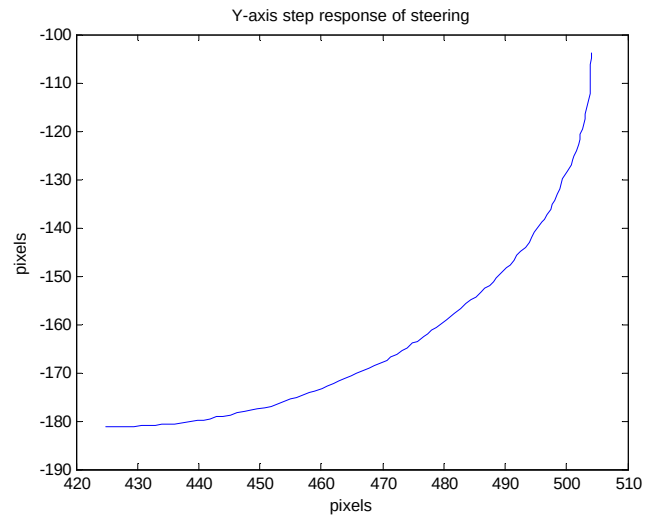


Figure 40: Orthographic view of step response

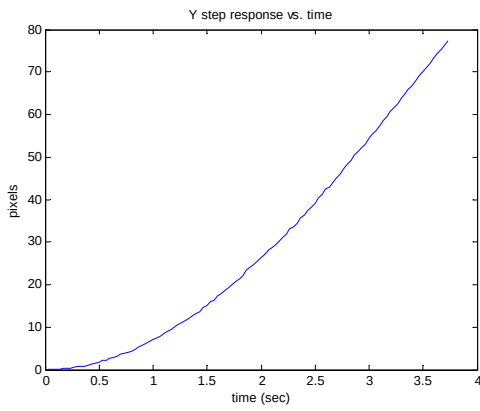


Figure 41: Step response as a function of time

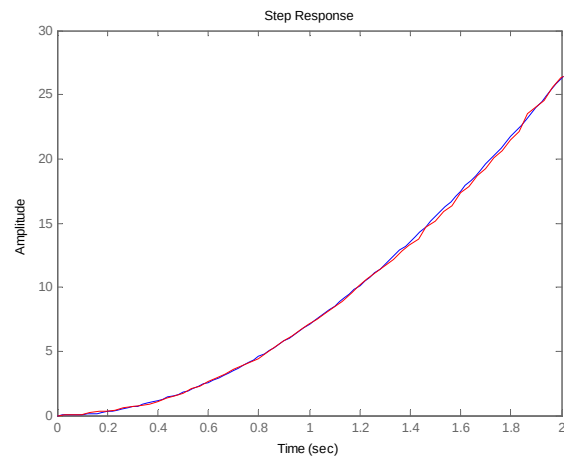


Figure 42: Modeled and Actual Step Responses (blue = modeled, red = actual)

Third Approach: Full State Linear Feedback

Although not actually used in our final design, this section covers an implementation of a feedback design for the non-holonomic robot using exact feedback linearization, rather than approximate feedback linearization. The advantages are an exponential convergence rate at the cost of using many more integrators and a more complex design, as well as a loss of intuition for the gain constants.

The model, as implemented in Simulink, is shown below:

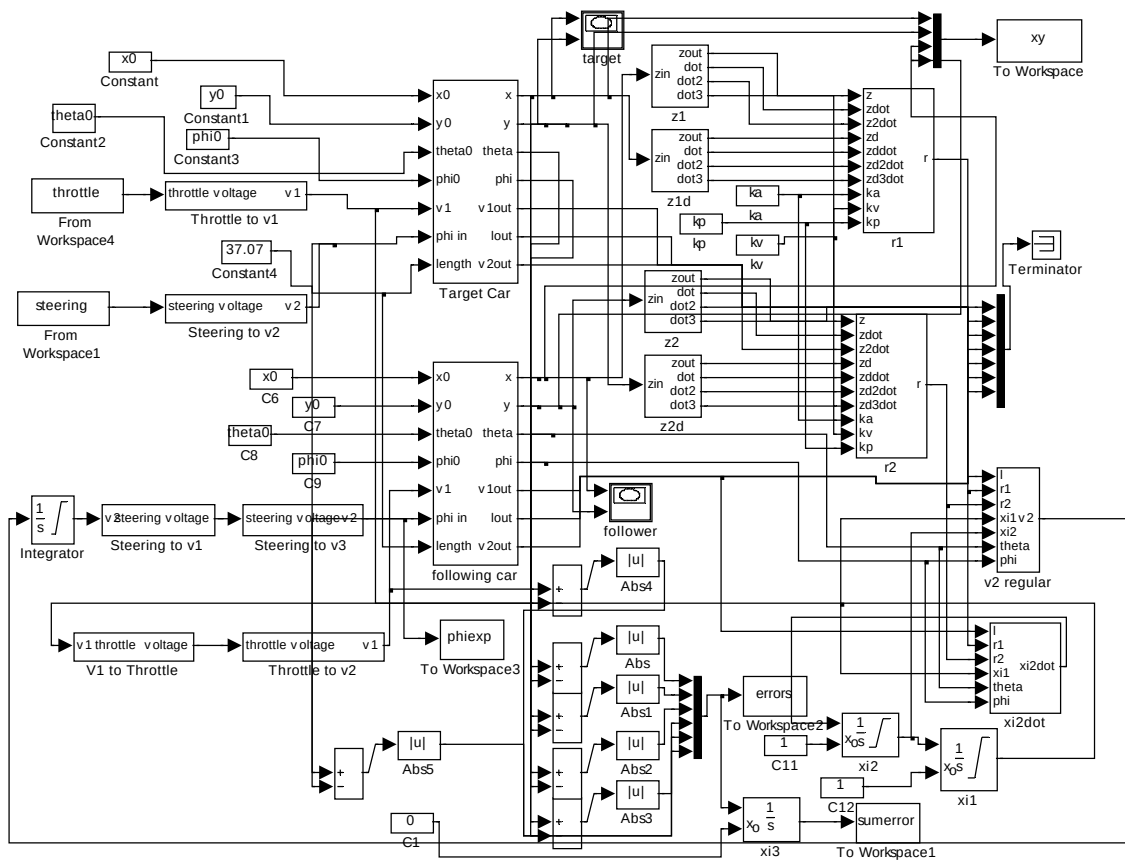


Figure 43: Simulink implementation of full state exact linear dynamic feedback controller

The target car merely outputs the path coordinates, here designated by (x_d, y_d) . Using the equations obtained from De Luca, Samson, “Feedback Control of a Nonholonomic Car-Like Robot,” p. 199-207, we implemented the feedback controller.

The following car has inputs calculated through the following equations for a system in chained form:

$$\begin{aligned} u_1 &= x_1 \\ u_2 &= (r_2 - x_3 r_1 - 3x_2 x_1 x_2) / x_1^2 \\ \dot{x}_1 &= x_2 \\ x_2 &= r_1 \end{aligned}$$

with

$$r_i = \ddot{z}_{di} + k_{di}(\ddot{z}_{di} - \ddot{z}_i) + k_{vi}(\dot{z}_{di} - \dot{z}_i) + k_{pi}(z_{di} - z_i), i=1,2$$

$$\begin{aligned} z &= \begin{bmatrix} x_1 \\ x_4 \end{bmatrix} \\ \dot{z} &= \begin{bmatrix} x_1 \\ x_3 x_1 \end{bmatrix} \\ \ddot{z} &= \begin{bmatrix} x_2 \\ x_2 x_1^2 + x_3 x_2 \end{bmatrix} \end{aligned}$$

Equation 29: Full State Linearization Equations for Car in Chained Form

However, similar design steps can be taken with the original, non-chained form car model, yielding the following equations

$$\begin{aligned} v_1 &= x_1 \\ v_2 &= \frac{-3x_2 \cos^2 f \tan f}{x_1} - \frac{\ell r_1 \cos^2 f \sin q}{x_1^2} + \frac{\ell r_2 \cos^2 f \sin q}{x_1^2} \\ \dot{x}_1 &= x_2 \\ \dot{x}_2 &= \frac{x_1^3 \tan^2 f}{\ell^2} + r_1 \cos q + r_2 \sin q \end{aligned}$$

with

$$r_i = \ddot{z}_{di} + k_{di}(\ddot{z}_{di} - \ddot{z}_i) + k_{vi}(\dot{z}_{di} - \dot{z}_i) + k_{pi}(z_{di} - z_i), i=1,2$$

Equation 30: Full State Linearization for Car in non-chained (regular) form

Thus yielding the following relationships for non-chained form:

$$\begin{aligned} z &= \begin{bmatrix} x \\ y \end{bmatrix} \\ \dot{z} &= \begin{bmatrix} x_1 \cos q \\ x_1 \sin q \end{bmatrix} \\ \ddot{z} &= \begin{bmatrix} -x_1^2 \tan f \sin q / \ell + x_2 \cos q \\ x_1^2 \tan f \cos q / \ell + x_2 \sin q \end{bmatrix} \end{aligned}$$

Using the two additional dummy variables, x_1 and x_2 , as input commands, as well as u_1 and u_2 , we are able to produce four input dimensions for the four respective states, x, y, q , and f . It is well known in robotics that, if the number of generalized coordinates equals the number of input commands, one can use a nonlinear static (i.e. memoryless) state feedback law in order to transform exactly the nonlinear kinematics/dynamics into a linear system.¹⁵ Thus, using these new feedback equations as described by Equation 30, we have implemented the system into Simulink, without having the hassle of case-conditions and state memory.

After a model was established and checked against real data and results, the dynamic feedback algorithm had to be tested with the new model. First, the real model placed upper and lower saturation limits on v_1 and v_2 . Thus, the growth rates and starting conditions for x_1 and x_2 , which can be taken as the amount of discomfort in x - and y -error, would have to be limited and occasionally reset if values went too negative or positive (since they determine the growth or decay rate of other state variables). Also, the values for the feedback coefficients, k_{ai}, k_{vi}, k_{pi} that scale the error in $(\ddot{z}_{di} - \ddot{z}_i), (\dot{z}_{di} - \dot{z}_i), (z_{di} - z_i)$, needed to be found. Taking the Laplace transform and solving for the closed-loop poles, one finds that the k -values are determined by setting the following roots to zero (satisfying the Routh-Hurwitz criterion for stability):

$$l^3 + k_{ai} l^2 + k_{vi} l + k_{pi}, i = 1, 2$$

Thus, a MATLAB scripting program was run that varied the following factors:

- 1.) Upper saturation limit for x_1 [10 to 150]

- 2.) Lower saturation limit for x_1 [-10 to 0]
- 3.) Upper saturation limit for x_2 [10 to 70]
- 4.) Lower saturation limit for x_2 [-10 to 1]
- 5.) Location of closed loop pole, k , so $k_{ai} = 3k, k_{vi} = k * k_{ai}, k_{pi} = k^3$ [-0.25 to -9]

For each iteration of these values, a following car attempted to track the following XY-trajectory, produced over 61.1 seconds in LabView, tracking a real car.

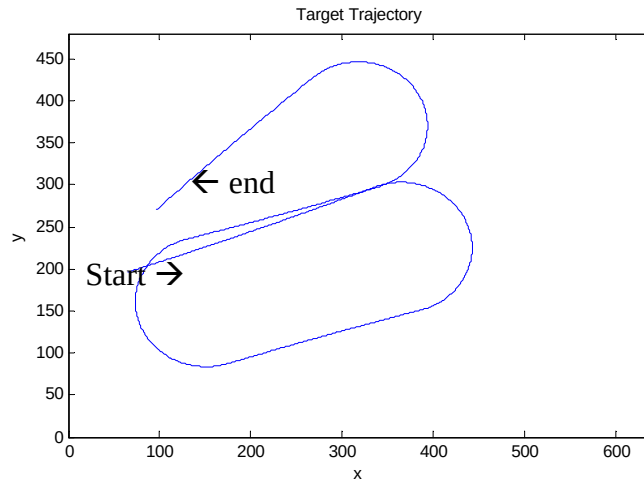


Figure 44: Target Trajectory

The target trajectory of Figure 44 was generated using real data captured from LabView, thus providing a completely feasible trajectory data on a 640x480 pixel image. With a sample setting of, say, [10 -9 10 -6 2] for [Upper saturation limit for x_1 , Lower saturation limit for x_1 , Upper saturation limit for x_2 , Lower saturation limit for x_2 , closed loop pole, k]. The resulting trajectory looked like the green path in Figure 45 below. Thus, as can be seen, although the algorithm guarantees exponential convergence, the extra constraints on v_1 and v_2 worsen the performance. However, many trials were run and each of the errors in state variables, x, y, q , and f , were integrated over the entire time, to see how much “raw” error resulted.

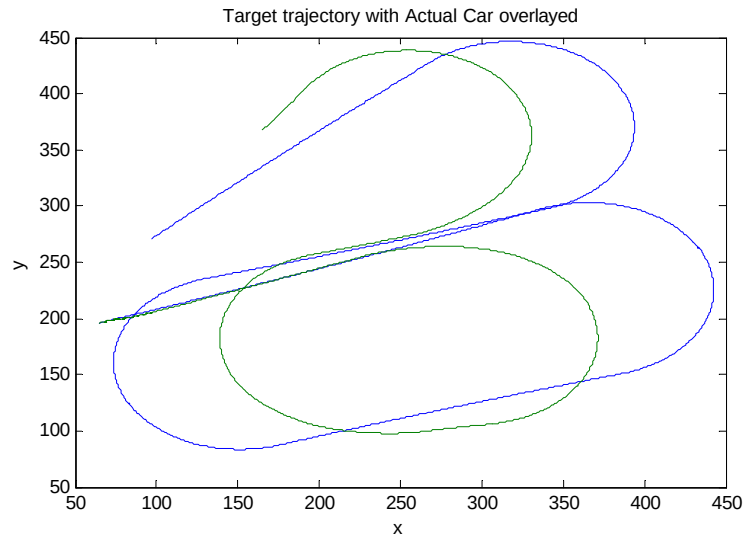


Figure 45: Target Trajectory with Realization superimposed

Some initial results are as follows:

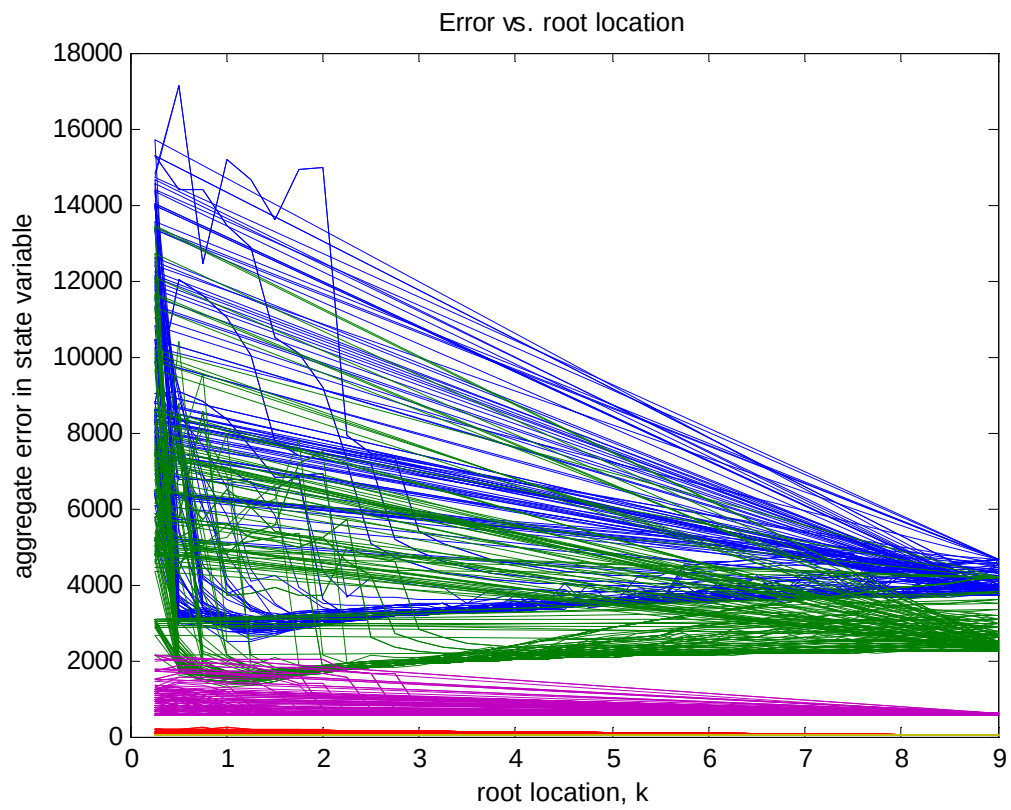


Figure 46: Errors vs. Root Location, k

Each of the four state variables' aggregate error is shown in a different color, with $\int \tilde{x} dt$, $\int \tilde{y} dt$, $\int \tilde{q} dt$, and $\int \tilde{f} dt$, shown by the blue, green, red, and purple streaks, respectively. Each error, integrated over the entire time interval, decreases as the root is moved out further on the left-hand plane. Thus this would suggest that dampening the system (moving the closed loop poles away from 0) would aid system performance, since the system is attempting to adjust too quickly, restricted too much by the v_1 and v_2 variables.

In addition to raw error in state variables, the input to v_2 (steering) was also tracked through the experiments, since some deceptively well-performing algorithms had v_2 inputs that looked something like the results in Figure 47. During such trials the steering input, v_2 , had to be changed at a very frequent rate to approximate what the car would physically realize. Since, of course, the turn doesn't take an infinitely short time (which is true of the real-life car), obviously a non-well behaved v_2 input would result in the system having to compensate even more than in the Simulink model. However, with different values of the feedback constants and saturation limits chosen, a more well-behaved v_2 input can be chosen, which usually results in better performance overall, as seen in Figure 48. Although initially the error results in control saturation at 0 or 5V, the transient response becomes smooth.

A near optimal input was found at [-3, 30, -3, 100, 3]. The resulting v_2 input vs. time graph is plotted in Figure 49. The resulting x , y – trajectories for the target and following car are shown, which are almost identical.

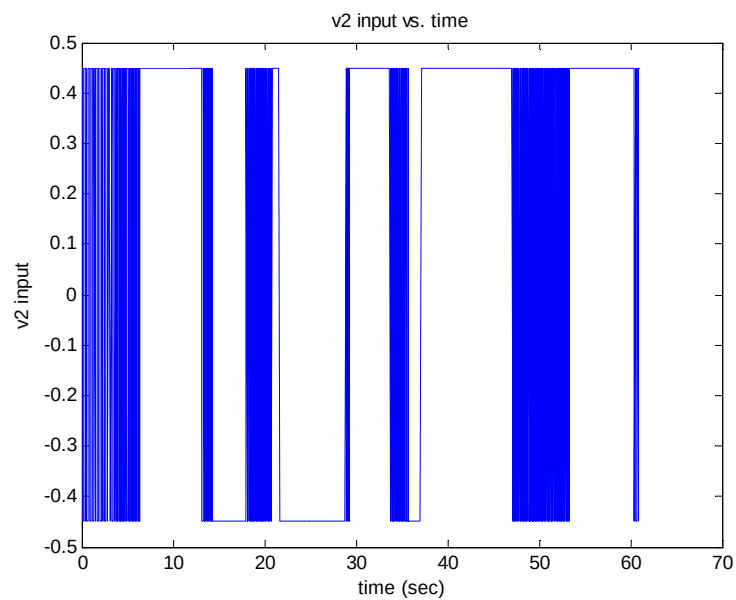


Figure 47: Non-well behaved v2 input

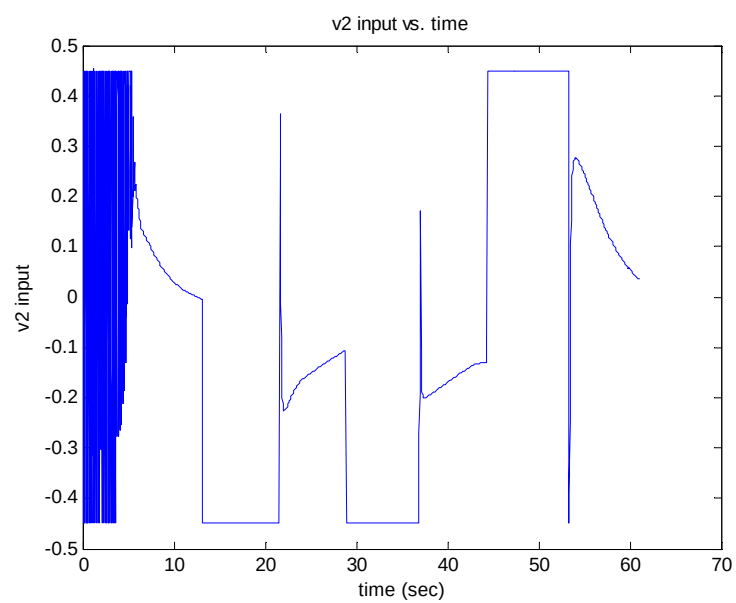


Figure 48: Well behaved v2 input

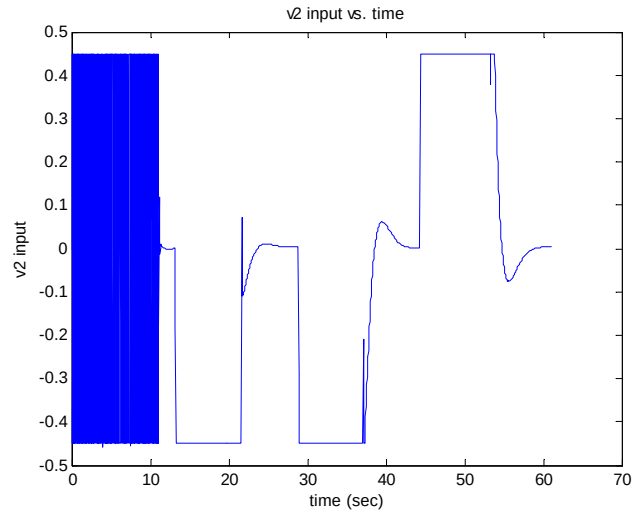


Figure 49: Near optimal v_2 vs. time input

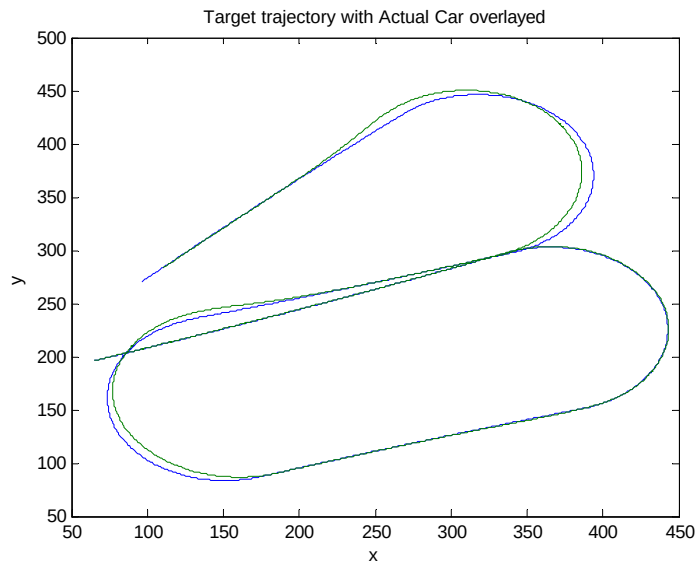


Figure 50: Trajectories for $[-3 \ 30 \ -3 \ 100 \ 3]$

An adaptive algorithm was used on this near-optimal data, to further refine the parameters. The algorithm:

- 1.) Randomly picks a state variable, x , y , q , or f , or a control value, v_1 or v_2 , to optimize.

- 2.) Perturbs each parameter, the lower or upper saturation limit for x_1, x_2 , or the root location, by a small amount, proportional to the current value of each.
- 3.) The integral of the errors, $\int \tilde{x} dt$, $\int \tilde{y} dt$, $\int \tilde{q} dt$, and $\int \tilde{f} dt$, was then computed for the target trajectory and appended to an aggregate matrix of error-sums.
- 4.) The trial with the minimum error of the chosen state/control variable is chosen to reset the parameters.
- 5.) Repeats N times.

Thus, this “random walk” optimization algorithm was used to generate an optimal set of parameters for the full-state model:

	x_1 upper lim	x_1 lower lim	x_2 upper lim	x_2 lower lim	Root
X	43.32	-1.32	16.82	-6.47	3.36
y	43.32	-1.32	16.82	-6.47	3.36
q	47.36	-2.69	24.13	-4.83	4.78
f	47.72	-1.66	6.71	-5.12	7.53
V1	44.64	-1.32	24.70	-6.50	3.45
V2	44.26	-1.80	9.39	-6.44	2.99

Table 2: Optimal parameter values to optimize each state variable's error

When using the first row's values, for instance, we see that the path following algorithm works with minimal deviation from the path:

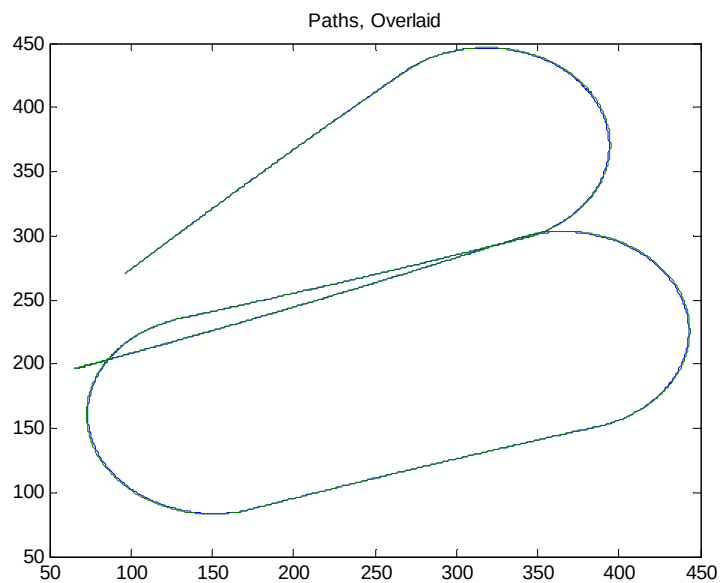


Figure 51: Path trajectories with optimal parameters

The same trajectory was repeated, except the following car was started off-trajectory, with $(x_0 + 300, y_0 - 180)$. Again, the overlaid graphs show quick convergence to the target path, in Figure 52.

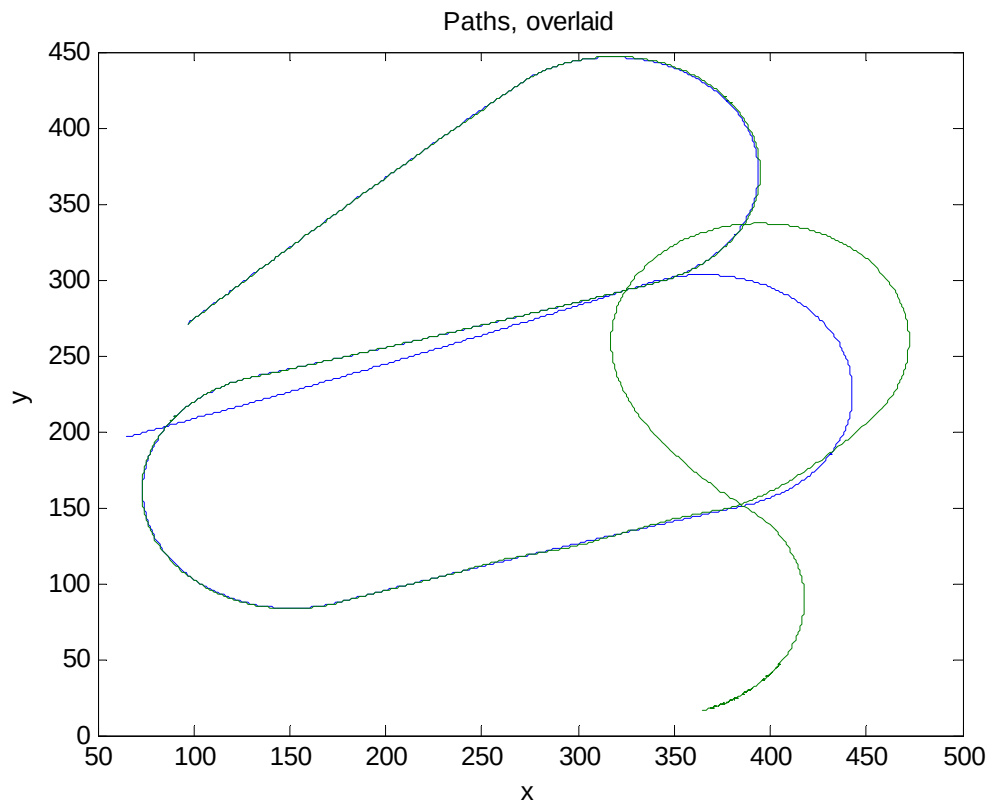


Figure 52: Off-trajectory simulation

Conclusion

Although well-suited for situations where there are large initial errors, the full-state linearized feedback algorithm lacks the intuitive appeal of the Linearized Original Coordinate form.

Chapter 6: Formation Driving

Our implementation for formation driving is to first define the target locations for each of the four vehicles, and then keep the vehicles in these positions. These positions can be one behind the other with a certain buffer distance, or all in a line, or in a V, or diamond, etc. Once these positions are determined, we use path planning to predict a trajectory from each car's current location to the desired position, and we use the controller to help it get there.

Traversing a Path

Before setting four vehicles off in formation, we began with one vehicle and verified that it could follow a path between two arbitrary points and orientations on the screen. Since the controller tracks a reference point, and changes to the control inputs are based on the error between the reference and the actual, it was necessary to continually feed the car a new point along the path, 30 times per second. In addition, it was necessary to generate the path with the distance between the points proportional to the desired speed. That way, as the error between the current position and the next reference point increased, the car would move faster along the path. Figure 53 shows a screenshot that has the vehicle being tracked and the target path for the vehicle, given a point and an orientation that we determined as the goal point. In this test, we began the vehicle off of the desired path to test that it would converge to the path over time and accurately end up at the final destination. Figure 54 shows the X and Y error as a function of time, and Figure 55 shows the orientation (theta) error as a function of time. Figure 56 and Figure 57 show the results for the same path and perturbation, but at a much higher speed. The convergence of the X-error to -10 in Figure 56 is not an error in the controller, but is an effect of the hardware. Since the path was horizontal, a negative error in the X direction would indicate that the car was behind its desired position. At a high desired velocity, the target point is moving at high speed, and the data shows that the car was unable to go fast enough to catch up to the target point. Most likely, the battery in the car was weak, reducing the top speed of the motor. Otherwise, the results show that the controller does get the car to converge to its target path and point, and that it is effective for controlling a car along a generated path.



Figure 53: Traversing a Path Screenshot

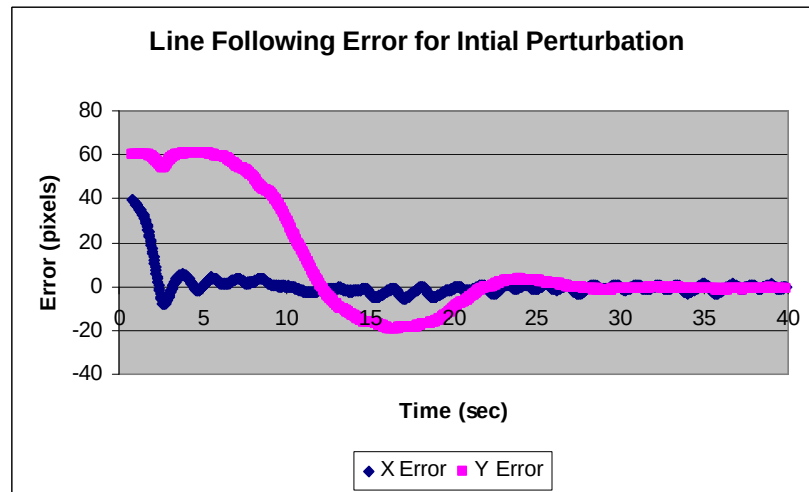


Figure 54: X and Y Errors for path following at low speed

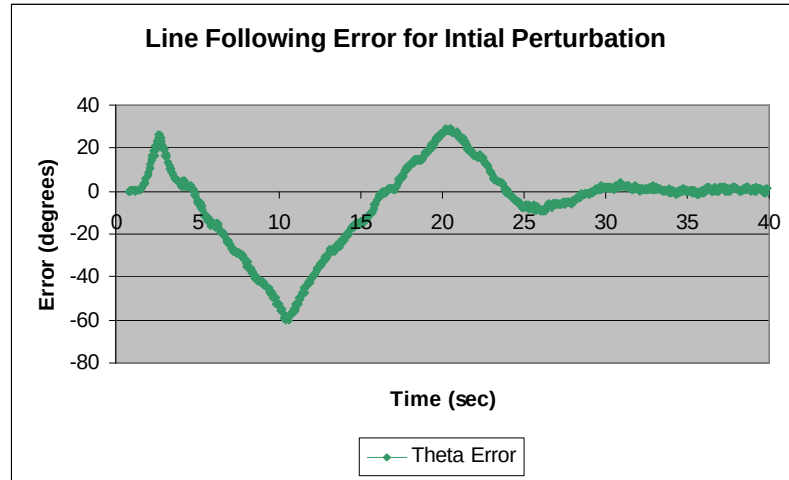


Figure 55: Theta error for path following at low speed

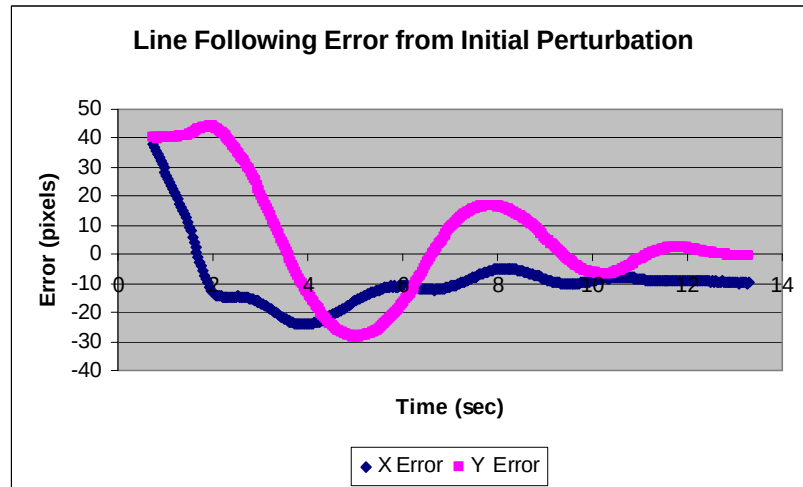


Figure 56: X and Y Errors for path following at high speed

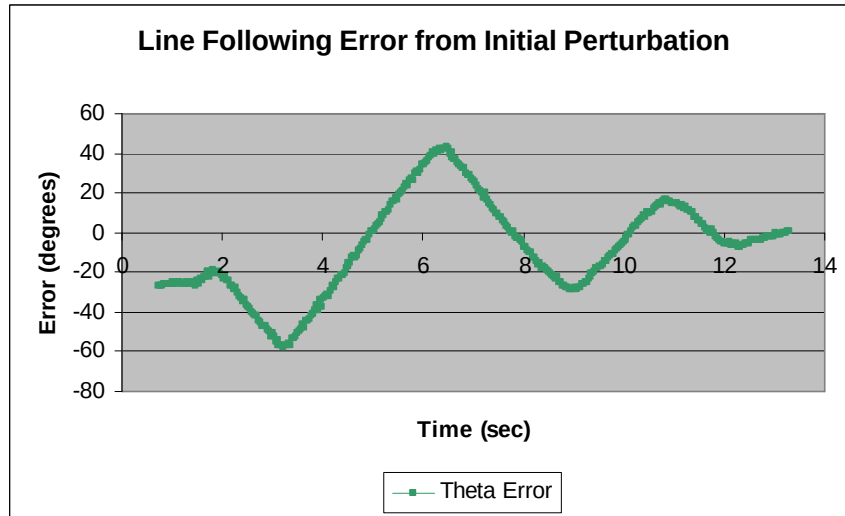


Figure 57: Theta error for path following at high speed

Successful traversal of a path was our first stepping stone on the way to formation driving. The next step was to create paths dynamically from moving destinations and use the path following algorithms to create formation driving.

Following the Leader

The next step on the way to formation driving was to create a formation with two cars: one leader and one follower. The goal was to build a system very similar to following a path, but to have the destination point be designated by the position of the leader car, as tracked by the vision algorithms. We drove the leader car with our own R/C controller, and the computer could only get information about it through the camera and tracking system.

Path Update

Now that the destination point was dynamic, it was necessary to dynamically update the desired path as well. See Figure 58 for an image of the first path generation between a leader car and a follower car.

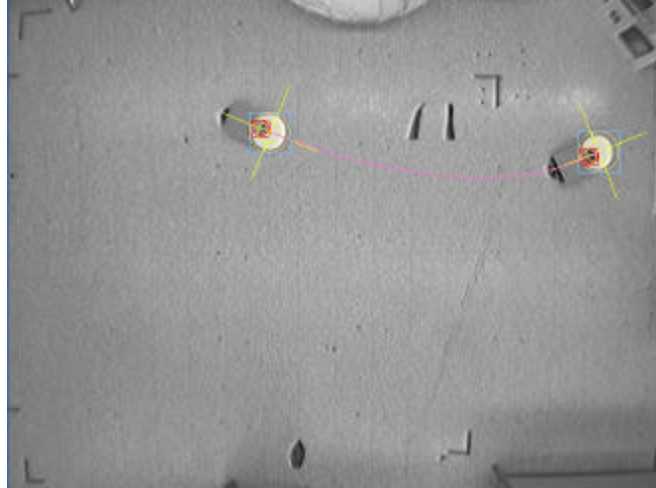


Figure 58: Dynamic path generation from one vehicle to another

Our first implementation was to update the target path every frame or every few frames. Updating very frequently solved one of our earlier problems with the cubic path planning, which was the problem of infeasible paths. Infeasible paths being paths that have a turn radius too small for the car to complete, given its physical limitations. Although the cubic path generator will always create a smooth path between the two points that preserves orientations at the start and end, only modifying the total length of the path will not always create a feasible path for the car to follow. Over very short distances and large changes in orientation, the actual car must do either a U-turn, or go completely around its target once in order to arrive with the right orientation. The general problem is that the vehicle is limited in its turn radius. However, when the controller is given a turn radius is too small, the car will still attempt to follow, and it turns to its greatest ability (saturation) in the correct direction. Then, with the path updating every few frames, the car effectively escapes this problem since the new path becomes more feasible as the car turns around.

Path Jump

Although we had solved the problem of path planning, the vehicles showed small errors when turning corners and the path was updated. When we set the update speed to every 15 frames (0.5 seconds) and the car was turning with full steering to one side, every time the path was reset, it would set the steering back to the middle. At first, we guessed that

the coasting of the car during the 30th of a second between calculations was causing problems, but we soon realized that it was also an effect of the path planning. Since the paths are planned to preserve the orientation of the vehicle at the beginning and end, it assumes that the velocity of the car is equal to its orientation, and by preserving orientations at the start and finish, it will also preserve velocities. However, when the car is turning, its velocity (direction) will not be equal to its orientation because its velocity has a perpendicular component in the direction of the turn. Steering angle is a direct measure of this discrepancy between orientation and velocity, but it is not an input to the path planning algorithm. Thus, the planned path for the car will start out going straight ahead for a few points, and end going straight ahead for a few points. This creates a great path when going between static points, but once everything is dynamic, it causes errors every time the path is updated when the car is not moving straight ahead. Therefore, we wrote in another factor, called “Jump”, which would jump ahead in the path by a certain number of points every time the path was updated. This would account for both the coasting between iterations of the program, and for the straight section of the cubic path right in front of the vehicle. We empirically found that 15 points ahead was a good jump distance, since it would give the vehicle enough perpendicular error to continue on its turning path, and enough distance ahead to outrun the coasting effects. We also found that the optimum time for updating the path was about every 15 frames (0.5 seconds).

Following Error

In order to follow a leader, one strategy is to continually follow a path to a point just behind the leader, and to set your speed equal to that of the leader. However, to account for disturbances in the system, changes to the leader’s speed, or increased travel time for the follower due to oscillations in the control, the follower will need to adjust its speed in order to maintain the same following distance.

For a fully optimized system, we would have an additional controller (perhaps PID), regulating the velocity of the follower car, based on the distance between the leader car and the following car. We did not have sufficient time to build such a controller, and we think this is a great place to begin for future work. However, we did implement a

rudimentary P (proportional) controller and a system that can remain stable when the error is low.

Our controller design consists of two parts. The first part is an algorithm that simply creates a new path every 15 frames, between the current position and the goal position (a certain number of pixels behind the leader). Since the velocity of the vehicle is determined by the distance between the points in the path, the desired velocity of the vehicle must be worked into the path planner. In order to maintain the correct distance between the leader and the follower car, we proportionally controlled the velocity of the following car, based on its absolute distance from the leader. Anywhere beyond 2-3 car lengths behind, and the follower would go full throttle in order to catch up. Once it was within half of a car length of its desired location, it would attempt to go a velocity equal to the speed of the leader car. This algorithm was very effective at minimizing the gross trailing error and catching up to the leader vehicle. However, when the error became small and the cars were very close together, this method did not work as well, due to the nature of the “jump” function. Jumping 15 frames ahead and overshooting is not a problem when the error is large and the following vehicle is already moving at a rate close to full throttle. However, in close proximity, this overshoot will cause the cars to collide, then go beyond the desired target for the next iteration, and force the follower to wait until the leader gets ahead again. Overall, the path regeneration with the jump function is not very good in the case of small errors.

Therefore, for the close situation, we designed another controller whose function was to smoothly follow the leader at a close distance. One of the problems with the first controller is that it has a slight discontinuity every time that it refreshes the path, due to coasting and since the path does not take steering angle into account. The strategy with the second controller was to create a continuous and smooth path (continuous first derivative) such that the car would travel smoothly when in close proximity. Instead of continually updating the path and beginning at the start each time, the new controller would append a path from the end of the current path to the current target position. This way, the following car would follow almost exactly the path that the leader had taken. The velocity for these paths was exactly equal to the velocity of the leader when the path was generated, so it was most effective when following closely. But, if the goal was to

catch the leader as quickly as possible, this algorithm was inefficient, since it would follow the path that the leader had taken, instead of cutting corners to catch up.

To take advantage of the strengths of each of the controllers in themselves, we implemented a hybrid system of the two. The error variable that we monitored was the distance between the following car and its desired position, behind the leader. At large error, the program uses the first controller (path regeneration) to travel at high speed and along the shortest path to reach its goal location. Then, once it gets within about half of a car length from its desired location, the controller switches to the second controller (path appending) to create a smooth path right behind the leader. This way, it catches up to the leader using a proportional controller, but before it is able to overshoot, it tries to simply mimic the past actions of the leader, using our vehicle controller, which we have proven to be stable.

We tested this system by controlling the leader with our own R/C controller, and having the computer track two cars and control the follower. Overall, the performance was quite impressive, as is visible in our presentation videos.

Driving in Formation

Caravan

With a robust system in place for following a leader, formation driving was simply a scaling to three cars, following three different “leaders”. The first formation we tried was as formation where each car follows the one in front of it, creating a linear “caravan”. In this case, the human-controlled car remained the leader of the pack, but two of the cars were set to follow the cars directly in front of them, which were also being controlled by the computer.

One of the advantages of using path planning for formation driving is that it is very robust to large errors in the formation. The formation algorithm is very stable when the cars are close together, but it is also very good at building the formation, since each car has a full path laid out for it to follow to its destination.

To test the formation convergence, we first set out the cars far away from their final formation points, then simply drove the leader car and let them fall into formation.

Visual images from the overhead camera of the starting condition (with overlaid cross-hairs and paths), and the ending condition, with all the vehicles in formation, are shown in Figure 59 and Figure 60, respectively. Figure 61 shows the distance error as a function of time, and it decays quickly to the target following distance of 80 pixels. The blue car is the most stable, and reaches the target following distance the most quickly, most likely because it was following the human driven vehicle. Our driving tended to be much smoother and less “jerky” than the computer controlled ones, and that tended to improve the following response. The errors between the leader car and the following car’s orientation (theta errors) are graphed in Figure 62. The results here are much more varied than with the distance error. First, at large distance errors, the theta error is generally irrelevant, since the vehicle will be following a path to its target, and the path is not likely to prescribe an orientation equal to that of the leader. Second, if the whole formation is turning, and there is some following distance between the cars, there will always be orientation error, and it is assumed. Nonetheless, theta error should never exceed 30 to 40 degrees in the close, stable case, so the large theta errors that we observe late in the test are a problem. Our best guess to the source of those errors is an improper “jump” value that causes the controller to read too far ahead for the given conditions, then output an unreasonable steering control value. Large theta error can result even if the steering value is set incorrectly for a few iterations. Overall, however, we have had good performance in the caravan formation mode, and the two controllers work together effectively.

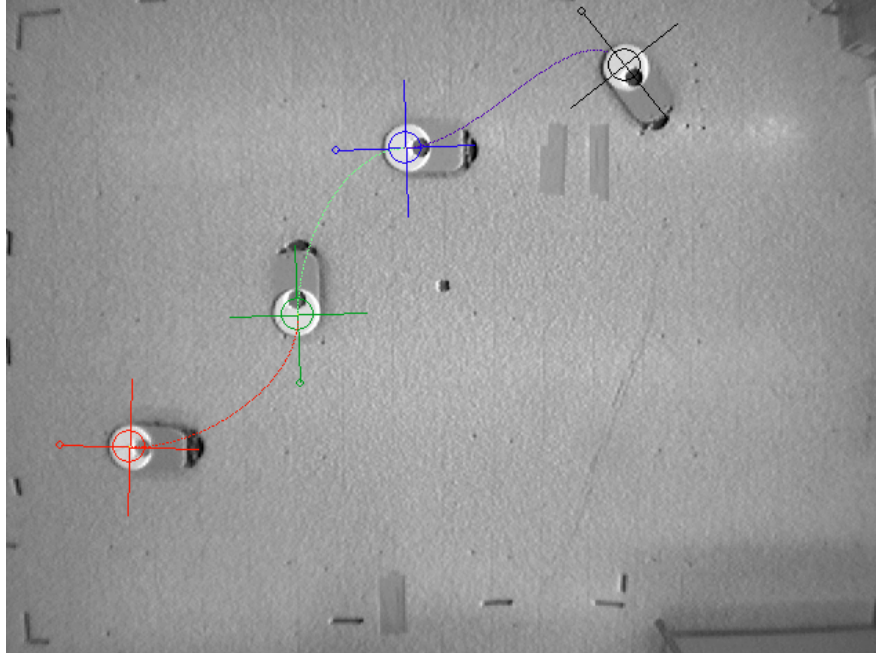


Figure 59: Convergence test for caravan formation - start

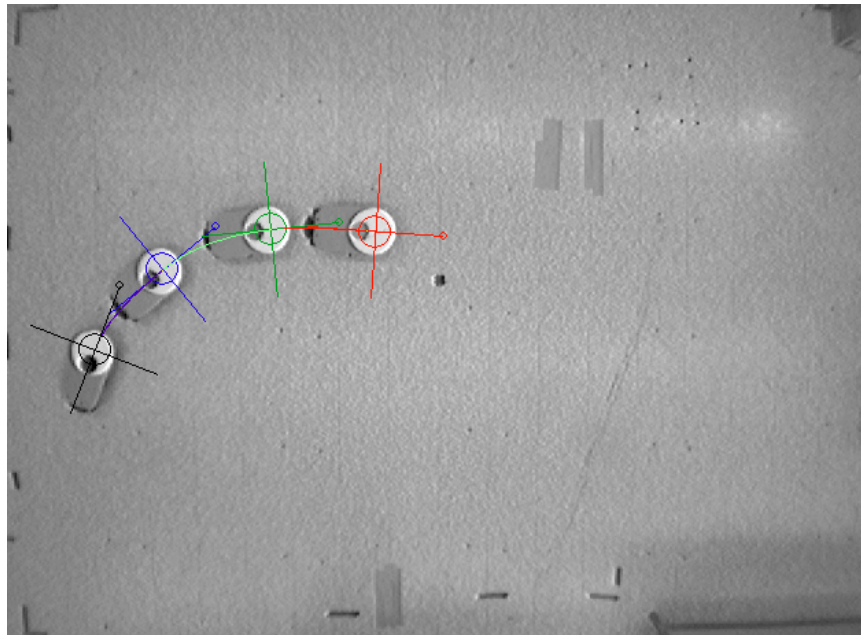


Figure 60: Convergence test for caravan formation - end

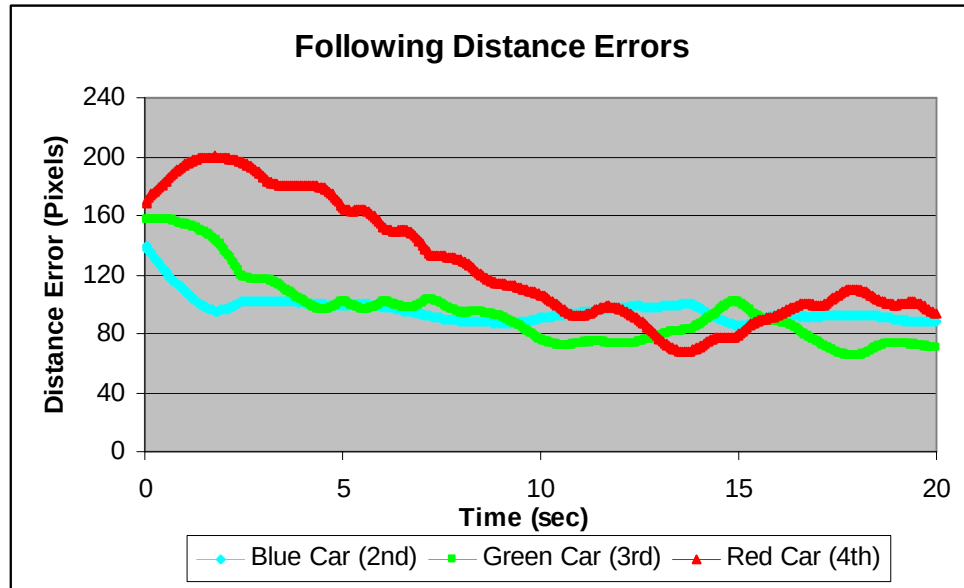


Figure 61: Distance error as a function of time for caravan formation

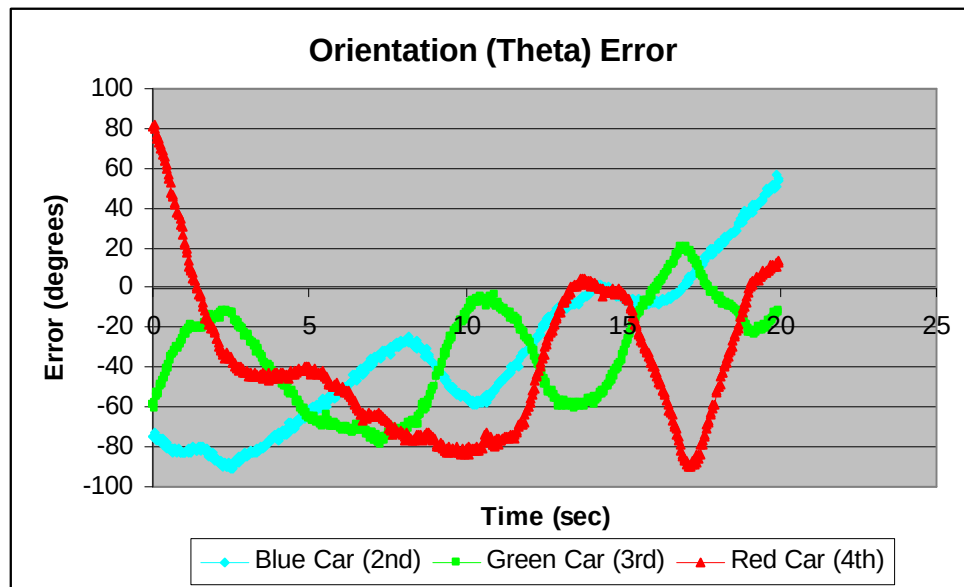


Figure 62: Orientation error as a function of time for caravan formation

Geese Formation

The second formation that we attempted was modeled after the V-formation of flying geese. Again, we had the leader car controlled by a human, and we kept the blue car following straight behind the leader. The red and the green cars were given points to the right and left, respectively. Figure 63 is an approximate view of the formation, with the

colored lines showing the path from each vehicle's current position to its desired position in the formation. Like the caravan formation, the path planning strategy gives very good response for large initial errors. Figure 64 shows how the path planning algorithm enables the vehicles to get into a formation even with significant initial error.

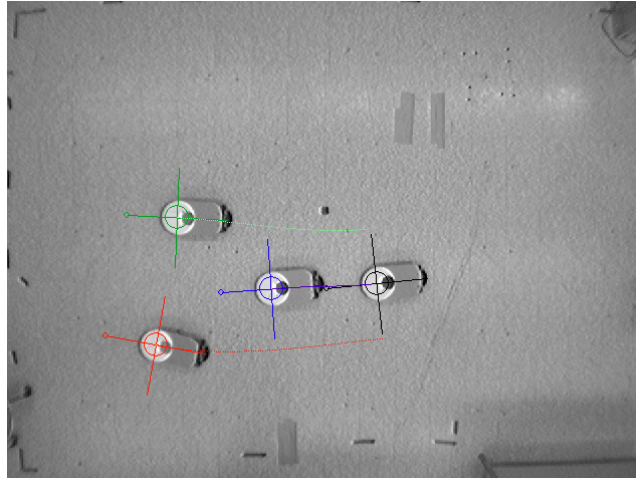


Figure 63: Geese Formation approaching steady-state

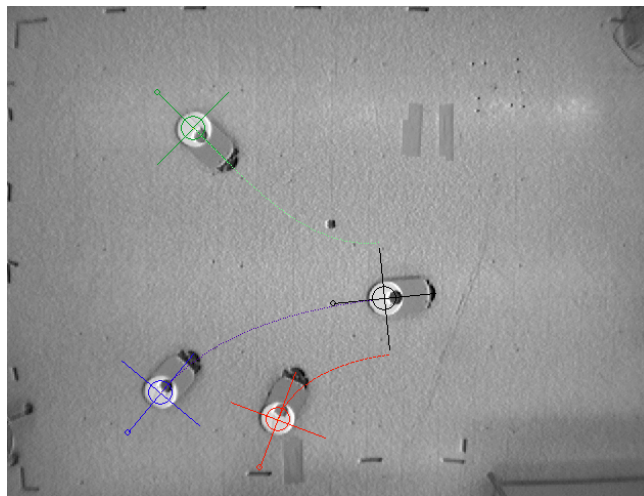


Figure 64: Geese formation with large initial error.

Unfortunately, we were unable to do significant testing of the geese formation with cars to the sides of the leader for a number of reasons. First, given the large size of the vehicles, their limited turning radius, the size of the classroom (H121), and the limited viewable area of the camera, it would be difficult to complete a full circle with the geese formation. The inside track vehicle is limited to the maximum turning radius of

the cars (which is about 80 pixels), and if each car is separated by 100 pixels, they would complete a full turn of 560 pixels in diameter. Our screen is only 640 pixels by 480 pixels, so there would be no way to do continuous testing of the geese formation. In addition, the formation controller must take into account a number of other variables when dealing with cars to the sides of the leader. When making turns, the inner car must make a tighter turn, so the leader should be limited to wider turns to prevent from crashing into the inside car. In addition, the outer car must not only follow a wider turn, but also travel a longer distance. Therefore, the controller must compensate for these differences and apply not the velocity of the lead car, but velocities based on the turning pattern. We did not have time to look into these areas of formation driving, but it would be a good place to start for future work.

Conclusions

Vehicles with nonholonomic constraints offer a new challenge to the typical control problem. To control rear-wheel drive vehicles effectively requires four state variables and two control variables. In addition, the nonholonomic constraints require path planning since errors cannot be minimized directly. This project has implemented a working system, in hardware and software that combines image processing, target tracking, path planning, and modern control theory to drive four vehicles in formation. However, we have not implemented the best or the most optimal path planning algorithms or control algorithms. Each of these alone could comprise an independent work project, but our goal was to build a working system that could also function as a test bed for improved algorithms in the future. With the hardware in place, the program will track and output the state variables of four individual vehicles. It is also capable of transmitting commands to three of those vehicles. Taking those inputs and outputs, future students can easily develop paths and control laws that will allow the cars to move faster, and more accurately. Our system is successful in itself, and also leaves a solid framework for future work in this field.

Future Work

We have built a LabView platform from which changes can be made quickly and efficiently by students in the future. These are some of our recommended future projects:

- Formations
 - A true controller that varies velocities to minimize the error between cars and their target points.
 - Improved algorithm formations, including velocity variation for the vehicles on the inside and outside track of turns.
 - Additional formations.
 - Smooth transitions between formations.
- Control Algorithm
 - Improve the car model by adding dynamic effects, including wheel slip for turning and acceleration. This will enable the controller to control faster moving vehicles.
 - Faster response time.
 - Control along a path and the use of predictive control, given a path to the destination.
- Path Planning
 - Create paths that are always viable for the vehicle and take the shortest route.
 - Create paths between static points, using forward and reverse.
 - Collision avoidance and obstacle avoidance.
- Work with a larger area or smaller cars to test wider formations.
- Use GPS instead of an overhead camera for large-scale applications.

Tips, Tricks, & Recommendations

LabView

- If possible, write your LabView functions in “Function Nodes” instead of writing Matlab code into LabView. It is extremely unnecessarily taxing on the computer to have to decode Matlab code when doing any simple calculations. For our algorithm to work, we also needed the refresh rate to remain at 30 frames per second, so we needed our VI to be clean and fast.
- Remember that LabView and Matlab and C all use different array indices. LabView, C, and Formula Nodes begin arrays at 0 while Matlab begins at 1.
- To ensure that all overlays appear on the image, and the image does not flicker, make sure to use IMAQ Image Copy, and create a new image for each unique image.
- To display the overlays on an outputted image (avi or jpeg), use the IMAQ function MergeOverlay to flatten the image.
- For the DisplayPoints overlay, displaying more than about 500 points slows down the system significantly, so try to limit the number of displayed points.

As always in MAE, KISS – Keep It Simple, Stupid.

Appendix A: Cubic Path Planning Algorithm as Instantiated into LabView

Function is written in Matlab:

%Given two points (x0,y0), a starting orientation (theta0),
ending conditions (xf,yf, thetalf), and a velocity_desired,
the function returns an array of points that are spaced at
the velocity desired, for an update every 1/30 second.

```
function out = cubicpath_lv(x0, y0, theta0, xf, yf, thetalf,
velocity_desired);
```

```
% Use absolute distance between the start and end positions
as the parameter for the length of the cubic path (time).
```

```
distance = sqrt((xf-x0)^2+(yf-y0)^2);
time = distance;
```

```
% Compute x values
```

```
dx0 = cos(theta0*pi/180);
dxf = cos(thetaf*pi/180);
```

```
a0x = x0;
a1x = dx0;
a2x = (3/(time^2))*(xf-x0)-(2/time)*dx0-(dxf/time);
a3x = (-2/time^3)*(xf-x0)+((dxf+dx0)/time^2);
```

```
% Compute y values
```

```
dy0 = sin(theta0*pi/180);  
dyf = sin(thetaf*pi/180);
```

```
a0y = y0;  
a1y = dy0;  
a2y = (3/(time^2))*(yf-y0)-(2/time)*dy0-(dyf/time);  
a3y = (-2/time^3)*(yf-y0)+((dyf+dy0)/time^2);
```

```
% Calculate the path for a constant velocity equal to
velocity_desired
```

```
% Use the known derivatives of the function at each point
to determine the correct time step of the function to
traverse the path at a constant velocity.
```

```
i=1;
time_step = 1;
t = 0;
while (t<time)
    out(i,1) = i*0.0333;
    out(i,2) = a3x*t^3+a2x*t^2+a1x*t+a0x;
    out(i,3) = a3y*t^3+a2y*t^2+a1y*t+a0y;
    i = i+1;

    dxdt =      3*a3x*t^2+2*a2x*t+a1x;
    dydt =      3*a3y*t^2+2*a2y*t+a1y;
    v = sqrt(dxdt^2+dydt^2);
    time_step = velocity_desired*0.03333/v;
    t = t + time_step;
end
```


Appendix B: Linear Control Algorithm as Instantiated into LabView

MATLAB linear control.vi

%rear wheel drive original coordinate linearized feedback

```
x = Con(1);
y = Con(2);
theta = mod(Con(3)*(2*pi)/360, 2*pi);
phi = mod(Con(4)*(2*pi)/360, 2*pi);

xd = Ref(1);
yd = Ref(2);
thetad = mod(Ref(3)*(2*pi)/360, 2*pi);
phid = mod(Ref(4)*(2*pi)/360, 2*pi);
v1r = Ref(5);
L = 37.07;
```

%Simulink optimized values

```
%k1 = 3.2802;
%k2 = 0.0291;
%k3 = 12.2030;
%k4 = 29.5395;
%up = 5.6636;
%down = 0.1302;

e1 = x-xd;
e2 = y-yd;
e3 = mod(theta+pi, 2*pi)-pi-(mod(thetad+pi, 2*pi)-pi);
e4 = mod(phi+pi, 2*pi)-pi-(mod(phid+pi, 2*pi)-pi);
e1r = cos(thetad)*e1+sin(thetad)*e2;
e2r = -sin(thetad)*e1+cos(thetad)*e2;
e3r = e3;
e4r = e4;
u1r = v1r;

k2 = 0.3;
%k2 = 0;

z1 = e1r;
z2 = e2r;
%z3 = e3r;
%z4 = e4r;
z3 = tan(e3r);
```

```

z4 = (tan(phi)-cos(e3r)*tan(phid))/(L*cos(e3r)^3)+k2*e2r;

con = 10;
k1 = 4.3;

%good set of values
k3 = 0.45;
k4 = 0.45;
k5 = -0.021;
%probably the best setting, or the one above
%going for gold...again
k3 = 1.05;
k4 = 1.55;
k5 = -0.05;

v1temp = -k1*z1;
v2temp = (-k3*z2/con-k4*z3-k5*z4);

w1 = abs(u1r)*v1temp;
w2 = abs(u1r)*v2temp;

%translates input voltage to a velocity value = pixels/sec
= v1

in = w1;
up = 84.0475;
uplow = 22.1006;
low = -77.7570;
lowhigh = -29.7136;

%from excel
backyint = 596.12;
backslope = 252.86;
forwardyint = 503.38;
forwardslope = 213.61;
    if((in <= 0) && (in >= lowhigh))
        out = (lowhigh+backyint)./backslope;
    elseif((in > 0) && (in <= uplow))
        out = (uplow+forwardyint)./forwardslope;
    elseif((in < lowhigh) && (in > low))
        out = (in+backyint)./backslope;
    elseif(in <= low)
        out = (low+backyint)./backslope;
    elseif((in > uplow) && (in < up))
        out = (in+forwardyint)./forwardslope;
    elseif(in >= up)

```

```

        out = (up+forwardyint)./forwardslope;
    end
    v1 = out;

    v2limit = 0.45;
    if((w2 > -v2limit)&&(w2 < v2limit))
        v2 = 4.876*w2+2.4807;
    end
    if(w2 <= -v2limit)
        v2 = 0;
    end
    if(w2 >= v2limit)
        v2 = 5;
    end
end

```

Appendix C: A Graphical Explanation of the LabView VI

Front Panel



Vision: Settings for the threshold values for the sizes of the tracking circles, and for the refresh rate of the color overlays on the image.

Run: Press *Find Car* to locate the vehicles, then *Order Shades* to identify them and display the crosshairs. Set the *Wideness of formation* and formation type before pressing "Go". To stop the cars from moving, press the *Let vehicle run?* button. Velocity sample time is the dt that it uses to calculate the velocity of the leader. The dt is greater than one to smooth out the noise in the data.

Colors Inputs and Indicators: *Path update* is the frequency (in frames) that each vehicle receives an updated path to follow. *Jump* is the number of points ahead that the program jumps when the path is refreshed. *Keep Away Distance* is the following distance (in pixels) between the leader and the one behind, or between the target points and

the cars behind.

The indicators give following data on each car. *Running?* Indicates whether the controller is sending new controls. When the *Regen mode?* light is on, the vehicle is far from its goal location, and it is regenerating the full path every path update. If the light is off, the vehicle is within $\frac{1}{2}$ of a car length of its desired location, and it is appending updated paths to its current path to maintain smooth performance.

LabView Image Description

Outside the Loop

The GSIM box ensures that all of the calculations inside are synchronized and lets us know if this is true, with the *sync* light. We do not explicitly use any GSIM functions in the VI.

The IEEE 1394 camera must be initialized before starting and closed at the end.

To save to Excel, we append an array during the program, so the array must be initialized outside the loop before we begin.

Vision System

The find car system resides at the top of the VI, with the chain of *Find Car* functions and *Black Out* functions, all controlled by the find car button.

Each of the icon “eyes” contains the car tracking algorithms. They receive input from the find car function, then continue to track the spots for the rest of the program.

On the far right is the display image window that takes the final image and outputs it to the screen.

Car Controllers

Each *Meta Car* takes the inputs of its current state variables and the state variables of its reference location, and outputs the results to the radio transmitters set by the numeric constants. (e.g. - 4, 5, etc.) *Meta Car* also keeps a number of variables in memory, and those come in and out via a giant cluster that is shift registered.

Within *Meta Car* resides the path planning algorithm, the velocity determining algorithm, the following algorithms, and the control algorithm.

“What formation?” gives the option of choosing various formations for the cars, and simply rearranges the destination points for each of the three controlled vehicles.

Miscellaneous

To capture data, there is a large array that accepts data while the program runs. It is currently accepting error data, which is the actual values minus the reference values. These are stored while the program is running and when the program is stopped, it prompts the user for a file name.

Reference List

1) Such projects include:

a.) Japan, "Demo 2000,"

http://www.ivsource.net/archivep/2000/sep/a000913_MELdemo.html#article.

b.) Dr. Ljubisa Vlacic, Griffith University, "Cooperative Autonomous Mobile Robots," financed by Australian Government, Japanese government and the Government of Germany.

c.) Alina Bejan Computer Science Department University of Iowa and Ramon Lawrence, IDEA Lab, University of Iowa, "Peer-to-Peer Cooperative Driving."

2) Demo 2000 of Japan has already coined a specific term for a hybrid system: "Dedicated Omni-purpose inter-vehicle communication Linkage Protocol for Highway automation, or DOLPHIN." Plans are to include a sensor suite consisting of differential GPS, machine vision, laser radar, and millimeter wave radar. No infrastructure components are used.

3) John J. Craig. *Introduction to Robotics: Mechanics and Control*. Chapter 7, pp191-200. Addison-Wesley. 1986.

4) Stuart Russel and Peter Norvig. *Artificial Intelligence - A Modern Approach*. 2nd ed., Prentice Hall Series, 2003, p.905.

5) Alessandro De Luca, Giuseppe Oriolo, Claude Samson, *Feedback Control of a Nonholonomic Car-Like Robot*. Jean-Paul Laumond (Editor). Laboratoire d'Analyse et d'Architecture des Systemes. Centre National de la Recherche Scientifique. 1998.

6) Claude Samson. *Commande*. INRIA, Route des Lucioles 06902 Sophia-Antipolis Cedex, France. October 30, 1998. (Available only in French).

7) Alessandro De Luca, Giuseppe Oriolo. "Control of Wheeled Mobile Robots: An Experimental Overview." *Dipartimento di Informatica e Sistemistica, Universita degli Studi di Roma*. "La Sapienza", Italy.

-
- 8) Giuseppe Oriolo, Alessandro De Luca. "Wheeled Mobile Robot Control Via Dynamic Feedback Linearization: Design, Implementation, and Experimental Validation." *IEEE Transactions on Control Systems Technology*. Vol. 10, No. 6, November 2002.
- 9) Claude Samson. "Path following and time-varying feedback stabilization of a wheeled mobile robot," *2nd Int. Conf. on Automation, Robotics and Computer Vision*. Singapore, 1992.
- 10) John J. Craig. Introduction to Robotics: Mechanics and Control. Chapter 7, pp191-200. Addison-Wesley. 1986.
- 11) Stuart Russel and Peter Norvig. *Artificial Intelligence - A Modern Approach*, 2nd ed., Prentice Hall Series, 2003, p.905.
- 12) Alessandro De Luca, Giuseppe Oriolo, Claude Samson. "Feedback Control of a Nonholonomic Car-Like Robot." Jean-Paul Laumond (Editor). Laboratoire d'Analyse et d'Architecture des Systemes. Centre National de la Recherche Scientifique. 1998.
- 13) Claude Samson. "Commande." INRIA, Route des Lucioles 06902 Sophia-Antipolis Cedex, France. October 30, 1998. (Available only in French).
- 14) Alessandro De Luca, Giuseppe Oriolo, "Control of Wheeled Mobile Robots: An Experimental Overview," Dipartimento di Informatica e Sistemistica, Universita degli Studi di Roma "La Sapienza", Italy.
- 15) Alessandro De Luca, Giuseppe Oriolo, Claude Samson, *Feedback Control of a Nonholonomic Car-Like Robot*. Jean-Paul Laumond (Editor). Laboratoire d'Analyse et d'Architecture des Systemes. Centre National de la Recherche Scientifique. 1998.
- 16) "Controller Design." <http://decision.csl.uiuc.edu/~toussant/report16/node3.html>
- 17) E-mail correspondence with Claude Samson. Many of the incorrect formulas originally presented in [5] have been corrected here. Also, the methods on determining "well-suited" feedback constants are being published for the first time in this article.
- 18) E-mail correspondence with Claude Samson: Claude.Samson@sophia.inria.fr, msyang@princeton.edu
- 20) Yutaka J. Kanayama and Bruce I. Hartman. "Smooth Local-Path Planning for Autonomous Vehicles." *The International Journal of Robotics Research*. Vol. 16, No. 3, June 1997 pp263-283.

21) Bryan Nagy and Alonzo Kelly. "Trajectory Generation for Car-like Robots Using Cubic Curvature Polynomials." *Field and Service Robots 2001*.
http://www.ri.cmu.edu/people/nagy_bryan_ext.html



Figure 66: Mike and Dave - Formation Driving at its best